

TOYOTA-32 BİT

FİNANS PORTALI PROJESİ

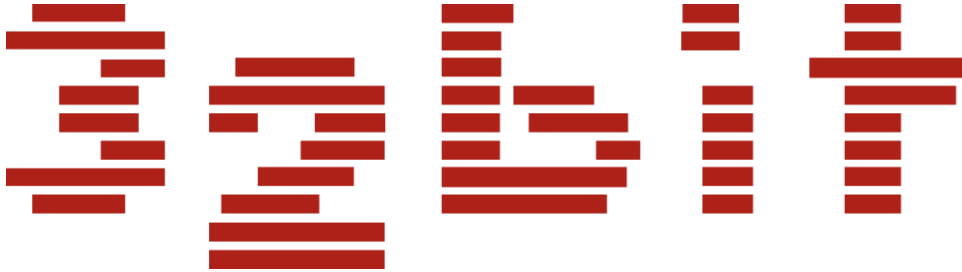
TEKNİK ANALİZ DÖKÜMANI (SDD)

Hazırlayan : Türkbey Yozcu

turbey.yozcu@ogr.sakarya.edu.tr



2026



32Bit Finans Portalı — Teknik Tasarım Dökümanı (SDD)

İçindekiler

Doküman Kontrol Bilgileri

1. Giriş

- [1.1 Amaç](#)
- [1.2 Kapsam](#)
- [1.3 Tanımlar ve Kısaltmalar](#)
- [1.4 Referanslar](#)
- [1.5 Notasyon](#)
- [1.6 Doküman Genel Bakışı](#)

2. Tasarım Genel Bakışı

- [2.1 Paydaş İlgili Alanları \(Stakeholder Concerns\)](#)
- [2.2 Mimari Sürücüler ve Kalite Hedefleri](#)
- [2.3 Seçilen Tasarım Bakış Açıları \(IEEE 1016 Viewpoints\)](#)
- [2.4 Teknoloji Yığını \(Stack\) Özeti](#)

3. Mimari Görünüm (C4 Model)

- [3.1 Bağlam \(C4 — Level 1\)](#)
- [3.2 Konteyner \(C4 — Level 2\)](#)
- [3.3 Bileşen \(C4 — Level 3, Backend\)](#)
- [3.4 Clean Architecture Katmanları ve Bağımlılık Kuralı](#)

4. Backend Tasarımı

- [4.1 Katmanlı Yapı ve İstek Akışı](#)
- [4.2 Domain Modülleri Tasarımı](#)
- [4.3 Dış Veri Entegrasyonu ve Symbol Routing](#)
- [4.4 Chart Strategy Deseni](#)
- [4.5 Dayanıklılık \(Resilience\)](#)
- [4.6 Hata Yönetimi](#)
- [4.7 Önbellekleme ve Zamanlanmış İşler](#)
- [4.8 Konfigürasyon ve Secret Yönetimi](#)
- [4.9 OOP Prensipleri ve Tasarım Desenleri](#)

5. Frontend Tasarımı

- [5.1 Uygulama Mimarisi](#)
- [5.2 Yönlendirme ve Sayfa Haritası](#)
- [5.3 Durum Yönetimi ve Veri Çekme Katmanı](#)
- [5.4 Grafik Bileşenleri Mimarisi](#)
- [5.5 Tema Sistemi](#)
- [5.6 Uluslararasılaştırma \(i18n\) Mimarisi](#)

6. Veri Tasarımı

- [6.1 Mantıksal Veri Modeli \(ER\)](#)
- [6.2 Veritabanı Şeması](#)
- [6.3 Veri Bütünlüğü Kuralları](#)
- [6.4 Şema Versiyonlama \(Migration\) ve Tohum \(Seed\)](#)
- [6.5 Önbellek \(Redis\) Tasarımı](#)

7. Arayüz / API Tasarımı

- [7.1 REST API Sözleşmesi](#)
- [7.2 Kimlik Doğrulama Akışı \(OAuth2 / OIDC\)](#)
- [7.3 Önemli Etkileşim Senaryoları](#)
- [7.4 Dış Entegrasyon Sözleşmeleri ve Degrade](#)

8. Güvenlik Tasarımı

- [8.1 Kimlik ve Yetki Modeli](#)
- [8.2 Ban \(Hesap Askıya Alma\) Mekanizması — Çok Katmanlı](#)
- [8.3 CORS, Secret ve Taşıma Güvenliği](#)
- [8.4 Tehdit Modeli \(STRIDE\) ve ASVS](#)

9. Dağıtım (Deployment) Tasarımı

- [9.1 Konteynerizasyon](#)
- [9.2 Kubernetes Topolojisi](#)
- [9.3 Ağ, Ingress ve TLS](#)
- [9.4 Ölçekleme, Depolama ve Secret](#)
- [9.5 CI/CD Boru Hattı](#)

10. Gözlemlenebilirlik Tasarımı

- [10.1 Üç Sütun Mimarisi](#)
- [10.2 Bileşenler ve Kararlar](#)
- [10.3 Pano ve Uyarı Stratejisi](#)

11. Çapraz Kesen Konular (Cross-Cutting Concerns)

12. Tasarım Kararları (ADR Kayıtları)

13. Riskler

14. Tasarım Doğrulama (V&V)

- [14.1 Test Stratejisi](#)
- [14.2 Doğrulama Yaklaşımı](#)

15. Ekler

- [Ek A — Teknoloji Versiyon Matrisi](#)
- [Ek B — Paket / Klasör Haritası](#)
- [Ek C — Açık Teknik Konular](#)
- [Ek D — 32Bit İster Uygunluk Matrisi](#)

Doküman Kontrol Bilgileri

Alan	Değer
Doküman Adı	32Bit Finans Portalı — Yazılım Tasarım Tanımı
Doküman Türü	Teknik Tasarım Dökümanı (SDD — Software Design)

	Description)
Sürüm	1.0 (Taslak)
Tarih	2026-06-07
Hazırlayan	Türkbey Yozçu
Durum	Taslak
Uyulan Standart	IEEE 1016-2009 · ISO/IEC/IEEE 42010:2022 · C4 Model · OWASP ASVS
İlgili Doküman	32Bit Finans Portalı — Analiz Dökümanı (SRS)

1. Giriş

1.1 Amaç

Bu doküman, **32Bit Finans Portalı** yazılımının **nasıl tasarlandığını ve inşa edildiğini** IEEE 1016-2009 (Software Design Description) standardına uygun biçimde tanımlar. Analiz Dökümanı (SRS) sistemin **ne yapması gerektiğini** (problem uzayı) tanımlarken; bu doküman **çözüm uzayını** — mimari, bileşen tasarımı, veri tabanı, API, güvenlik, dağıtım ve gözlemlenebilirlik kararlarını — ele alır.

SRS (Analiz Dökümanı)	SDD (Bu Doküman)
NE? (problem uzayı)	NASIL? (çözüm uzayı)
Gereksinimler, iş kuralları	Mimari, bileşenler, veri/şema tasarımı
Kabul kriterleri	Tasarım kararları (ADR), arayüz sözleşmeleri
İş/kullanıcı bakışı	Geliştirici/mimar/DevOps bakışı

Hedef kitle: Geliştiriciler, yazılım mimarı, DevOps/operasyon ve bakım ekibi.

1.2 Kapsam

Bu doküman aşağıdaki tasarım alanlarını kapsar:

Bölüm	Tasarım Alanı
2–3	Tasarım genel bakışı ve mimari görünüm (C4 model)
4–5	Backend ve frontend bileşen tasarımı
6	Veri tabanı / kalıcılık tasarımı
7	API sözleşmesi ve etkileşim akışları
8	Güvenlik tasarımı (kimlik, yetki, ban, tehdit modeli)
9–10	Dağıtım (Kubernetes/CI-CD) ve gözlemlenebilirlik tasarımı
11–14	Çapraz kesen konular, tasarım kararları (ADR), riskler, doğrulama

Kapsam dışı: Ürün/iş gereksinimleri, kullanıcı ihtiyaçları ve kabul kriterleri **SRS**'te tanımlıdır; burada tekrar edilmez, yalnızca referans verilir.

1.3 Tanımlar ve Kısaltmalar

Alan/iş terimleri için SRS §1.4'e bakınız. Tasarıma özgü terimler:

Terim	Açıklama
SDD	Software Design Description — Yazılım Tasarım Tanımı

C4	Context–Container–Component–Code mimari diyagram modeli
ADR	Architecture Decision Record — Mimari Karar Kaydı
Viewpoint	IEEE 1016 tasarım bakış açısı
Modüler Monolit	Tek dağıtılabılır uygulama içinde ayrı alan modülleri
DTO	Data Transfer Object
ORM/JPA	Object-Relational Mapping / Jakarta Persistence API
MapStruct	DTO–entity eşleme kod üretici
OIDC/JWT	OpenID Connect / JSON Web Token
SPI	Service Provider Interface (Keycloak eklenti arayüzü)
OTel	OpenTelemetry
Ingress / TLS	Kümeye giriş yönlendirmesi / Taşıma katmanı şifrelemesi
HPA / PDB / PVC	Horizontal Pod Autoscaler / Pod Disruption Budget / Persistent Volume Claim
CI/CD	Continuous Integration / Continuous Deployment
STRIDE	Tehdit modelleme çerçevesi (Spoofing, Tampering, ...)
ASVS	OWASP Application Security Verification Standard

1.4 Referanslar

No	Kaynak	Açıklama
1	IEEE 1016-2009 — Software Design Descriptions	Birincil standart (SDD yapısı)
2	ISO/IEC/IEEE 42010:2022 — Architecture description	Mimari tanımı / paydaş ilgi alanları
3	C4 Model (c4model.com)	Mimari görselleştirme yaklaşımı
4	OWASP ASVS	Güvenlik doğrulama standardı
5	32Bit Finans Portalı — Analiz Dökümanı (SRS)	Bağlı gereksinim dökümanı
6	Proje deposu: finance_portal, finance-frontend, k8s	Kaynak kod referansı

1.5 Notasyon

Mimari görünüm **C4 modeli** (Bağlam/Konteyner/Bileşen) ile; etkileşimler **UML sıralı (sequence)** diyagramlarıyla; veri modeli **Varlık-İlişki (ER)** diyagramıyla gösterilir. Diyagramlar **Mermaid** koduyla gömülebilir veya görsel olarak çizilir. Kod öğeleri monospace ile yazılır.

1.6 Doküman Genel Bakışı

Bölüm	İçerik
2	Tasarım genel bakışı: paydaş ilgi alanları, mimari sürücüler, teknoloji yığını
3	Mimari görünüm — C4 (bağlam, konteyner, bileşen, katmanlar)
4	Backend tasarımı (katmanlar, domain'ler, entegrasyon, dayanıklılık)

5	Frontend tasarımı (yapı, yönlendirme, durum, grafik, tema, i18n)
6	Veri tasarımı (ER, şema, bütünlük, migration, önbellek)
7	API tasarımı (REST sözleşmesi, kimlik akışı, etkileşim senaryoları)
8	Güvenlik tasarımı (kimlik/yetki, ban, CORS/TLS, tehdit modeli)
9	Dağıtım tasarımı (Docker, Kubernetes, ağ, CI/CD)
10	Gözlemlenebilirlik tasarımı (metrik/iz/log)
11	Çapraz kesen konular
12	Tasarım kararları (ADR kayıtları)
13	Riskler
14	Tasarım doğrulama (V&V)
15	Ekler

2. Tasarım Genel Bakışı

2.1 Paydaş İlgili Alanları (Stakeholder Concerns)

ISO/IEC/IEEE 42010 uyarınca, mimari paydaşların ilgi alanlarına yanıt verecek şekilde tanımlanır:

Paydaş	İlgi Alanı (Concern)	İlgili Bölüm
Yazılım Mimarı	Bütünsel yapı, modülerlik, teknoloji uyumu	§3, §4, §5
Geliştirici	Katmanlar, domain deseni, bileşen sınırları, test	§4, §5, §14
DevOps / Operasyon	Dağıtılabilirlik, ölçekleme, gözlemlenebilirlik	§9, §10
Güvenlik	Kimlik/yetki, ban, veri koruma, tehdit modeli	§8
Veri / DBA	Şema, bütünlük, migration, önbellek	§6
Son Kullanıcı	Performans, erişilebilirlik (dolaylı)	§7, §10
İş Sahibi (32Bit)	Düşük işletme maliyeti, sürdürülebilirlik, teslim	§2.2, §12
Bakım Ekibi	Anlaşılabilirlik, tasarım kararlarının gerekçesi	§11, §12

2.2 Mimari Sürücüler ve Kalite Hedefleri

Mimariyi şekillendiren başlıca sürücüler ve bunlara verilen tasarım yanıtları:

Sürücü	Tasarım Yanıtı	İlgili NFR (SRS)
Çok sayıda, ücretsiz ve kırılğan dış veri kaynağı	Periyodik @Scheduled senkronizasyon → Redis önbellek → controller okur; son-iyi/yedek değerler	NFR-REL-01
Yanıt süresi hedefi (P95 < 2 sn)	İstek anında dış API beklemeden	NFR-PERF-01

	önbellekten servis	
Güvenli kimlik ve kötüye kullanım kontrolü	Keycloak/OIDC + JWT; çok katmanlı ban (SPI + filtre + job)	NFR-SEC-*
Sürdürülebilir, anlaşılır kod tabanı	Modüler monolit: 19 alan modülü, katmanlı yapı	NFR-MNT-01
Düşük işletme maliyeti	Tek monolit; self-hosted çeviri (Lingva); ücretsiz API katmanları	İş hedefi
Operasyonel görünürlük	OpenTelemetry 3-sütun (metrik/iz/log)	NFR-OBS-*
Taşınabilir dağıtım	Docker imajları + Kubernetes (GKE)	NFR-PORT-01
Çok dilli, çok temalı arayüz	i18next (TR/EN) + tema sistemi (frontend)	NFR-USE-02

2.3 Seçilen Tasarım Bakış Açıları (IEEE 1016 Viewpoints)

IEEE 1016, tasarımın ihtiyaca göre seçilen "bakış açıları" ile anlatılmasını önerir. Bu dokümanda kullanılanlar:

Bakış Açısı (Viewpoint)	Açıklama	Bu Dokümanda
Context (Bağlam)	Sistemin dış dünya ile sınırları	§3.1
Composition (Bileşim)	Konteyner ve bileşenlere ayrıştırma	§3.2, §3.3
Logical (Mantıksal)	Katmanlar, modüller, sınıf/sorumluluk dağılımı	§4, §5
Dependency (Bağımlılık)	Katmanlar arası bağımlılık kuralı	§3.4
Information (Bilgi)	Veri modeli, şema, kalıcılık	§6
Interface (Arayüz)	API sözleşmesi, dış entegrasyon arayüzleri	§7
Interaction (Etkileşim)	Sıralı akışlar (auth, alarm, tool-calling)	§7.3
State Dynamics (Durum)	Alarm ve ban durum geçişleri	§4.7, §8.2
Deployment (Dağıtım)	Çalışma zamanı düğümleri, konteynerler	§9
Resource (Kaynak)	CPU/bellek/depolama, ölçekleme	§9.4

2.4 Teknoloji Yığını (Stack) Özeti

Backend

Katman / Amaç	Teknoloji
Dil / Çatı	Java 21, Spring Boot 3.3.9
Web / API	Spring Web (REST), springdoc-openapi (Swagger UI)
Kalıcılık	Spring Data JPA (Hibernate), PostgreSQL, Flyway (migration)
Önbellek	Spring Data Redis
Güvenlik	Spring Security, OAuth2 Resource Server (JWT), Keycloak Admin Client 24
Mesajlaşma / Arama	Spring Kafka, Spring Data Elasticsearch (OpenSearch)
Dizin	Spring Data LDAP (OpenLDAP)
E-posta	Spring Mail + Thymeleaf (şablon)
Eşleme / Yardımcı	MapStruct, Lombok, jsoup (scraping), Yahoo Finance API, commons-codec (TOTP)

Gözlemlenebilirlik	Micrometer (Prometheus), OpenTelemetry Java Agent + annotations, log4j2 (JSON)
--------------------	--

Frontend

Amaç	Teknoloji
Çatı / Derleme	React 19, Vite 8
Yönlendirme	react-router-dom 7
Veri çekme / durum	@tanstack/react-query 5, axios; Context (Preferences/Auth)
Grafikler	klinecharts, lightweight-charts, recharts
Stil / Tema	Tailwind CSS 4 (+ tema token'ları)
Çok dillilik	i18next, react-i18next, browser-languagedetector
Diğer	react-simple-maps, date-fns, jspdf + xlsx (dışa aktarım), lucide-react
Test	Vitest, Testing Library

Veri & Altyapı / Dış Servisler

Bileşen	Teknoloji / Servis
Veritabanı	PostgreSQL
Önbellek	Redis
Mesajlaşma	Apache Kafka
Log / Arama	OpenSearch
Kimlik	Keycloak (OIDC) + OpenLDAP
Çeviri / AI	Lingva (self-hosted), Gemini & Groq (LLM)
Gözlemlenebilirlik	OTel Collector, Tempo (iz), Prometheus (metrik), Grafana (pano)

Dağıtım

Amaç	Teknoloji
Konteyner	Docker
Orkestrasyon	Kubernetes (GKE Autopilot)
Sertifika / TLS	cert-manager
CI/CD	GitHub Actions

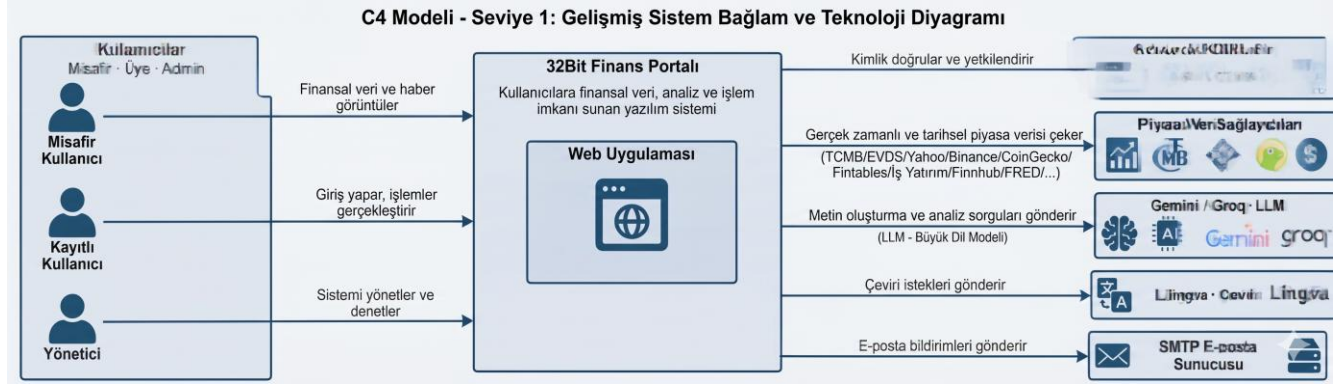
3. Mimari Görünüm (C4 Model)

Mimari, C4 modelinin üç düzeyinde (Bağlam → Konteyner → Bileşen) ve ayrıca katman/bağımlılık görünümüyle anlatılır.

3.1 Bağlam (C4 — Level 1)

Sistem, dış dünya ile sınırları açısından: kullanıcılar (misafir/kayıtlı/admin) tarayıcı üzerinden erişir; sistem kimliği Keycloak'a devreder, verisini çok sayıda dış sağlayıcıdan toplar (ayrıntı: SRS §2.1, §5.4).

— C4 Bağlam Diyagramı (Level 1)

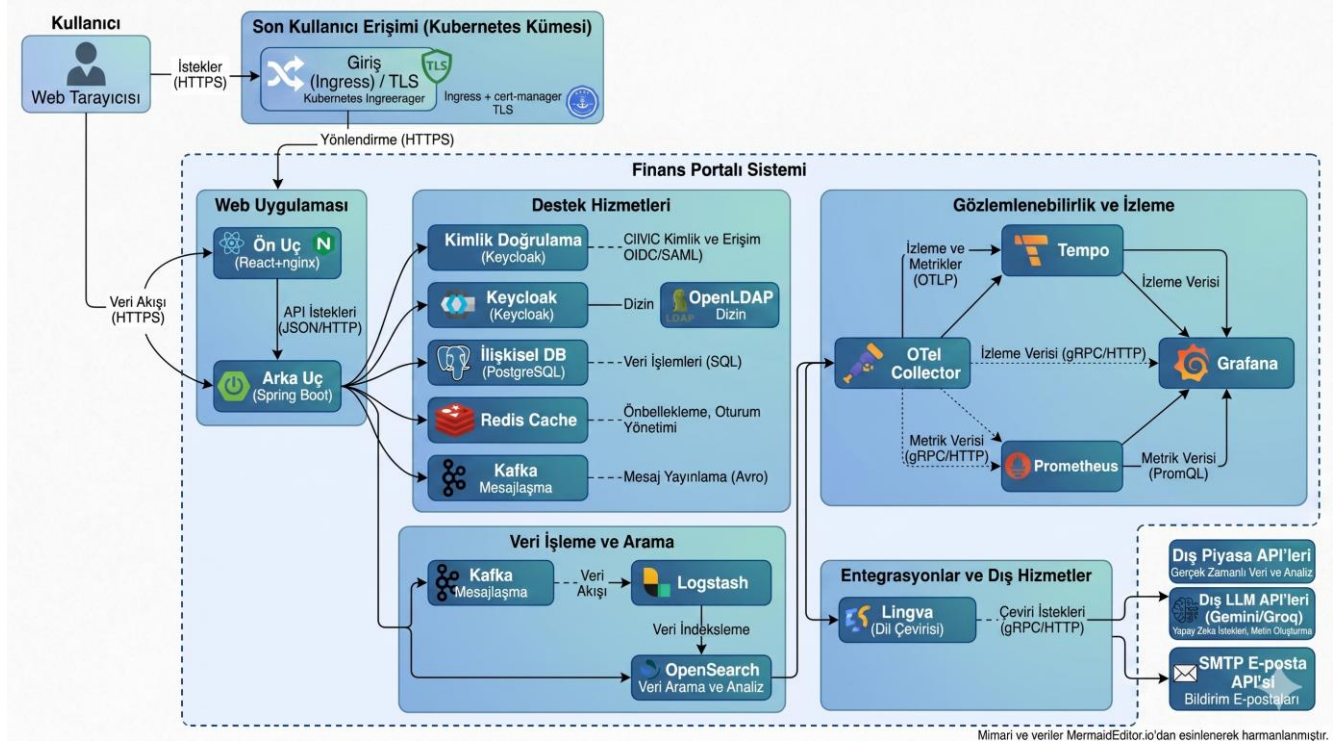


3.2 Konteyner (C4 — Level 2)

Sistem, **Docker imajları** olarak paketlenen birden çok konteynerden oluşur. Bileşenler birbirinden ayrı konteynerlerdir ve iki seviyede orkestre edilebilir: **yerde Docker Compose** (finance_portal/docker-compose.yml) ve **üretimde Kubernetes (GKE)**. Aşağıdaki tablo konteynerleri ve Kubernetes manifest karşılıklarını listeler:

Konteyner	Sorumluluk	Teknoloji	k8s Manifest
Frontend	React SPA sunumu	React/Vite + nginx	app/62-frontend
Backend	REST API, iş mantığı, senkronizasyon, sohbet	Spring Boot + OTel agent	app/61-backend
Keycloak	Kimlik/OIDC, 2FA, ban SPI	Keycloak	auth/42-keycloak
OpenLDAP (+ phpLDAPadmin)	Dizin servisi (+ yönetim arayüzü)	OpenLDAP	auth/40,41
PostgreSQL	Kalıcı veri	PostgreSQL	data/20-postgres
Redis	Önbellek	Redis	data/21-redis
Kafka (+ Zookeeper)	Asenkron olay / log akışı	Apache Kafka	messaging/31,30
Logstash	Kafka → OpenSearch log boru hattı	Logstash	messaging/32
OpenSearch (+ Dashboards)	Log deposu ve arama (+ görselleştirme)	OpenSearch	data/22, obs/54
OTel Collector	Telemetri toplama/iletme	OpenTelemetry Collector	obs/50
Tempo	İz (trace) deposu	Grafana Tempo	obs/51
Prometheus	Metrik toplama	Prometheus	obs/52
Grafana	Panolar / görselleştirme	Grafana	obs/53
Lingva	Haber TR↔EN çeviri	Lingva (self-hosted)	app/60-lingva
Ingress + cert-manager	Giriş yönlendirme + TLS sertifikası	GKE Ingress + cert-manager	network/70,71,72

— C4 Konteyner Diyagramı (Level 2)

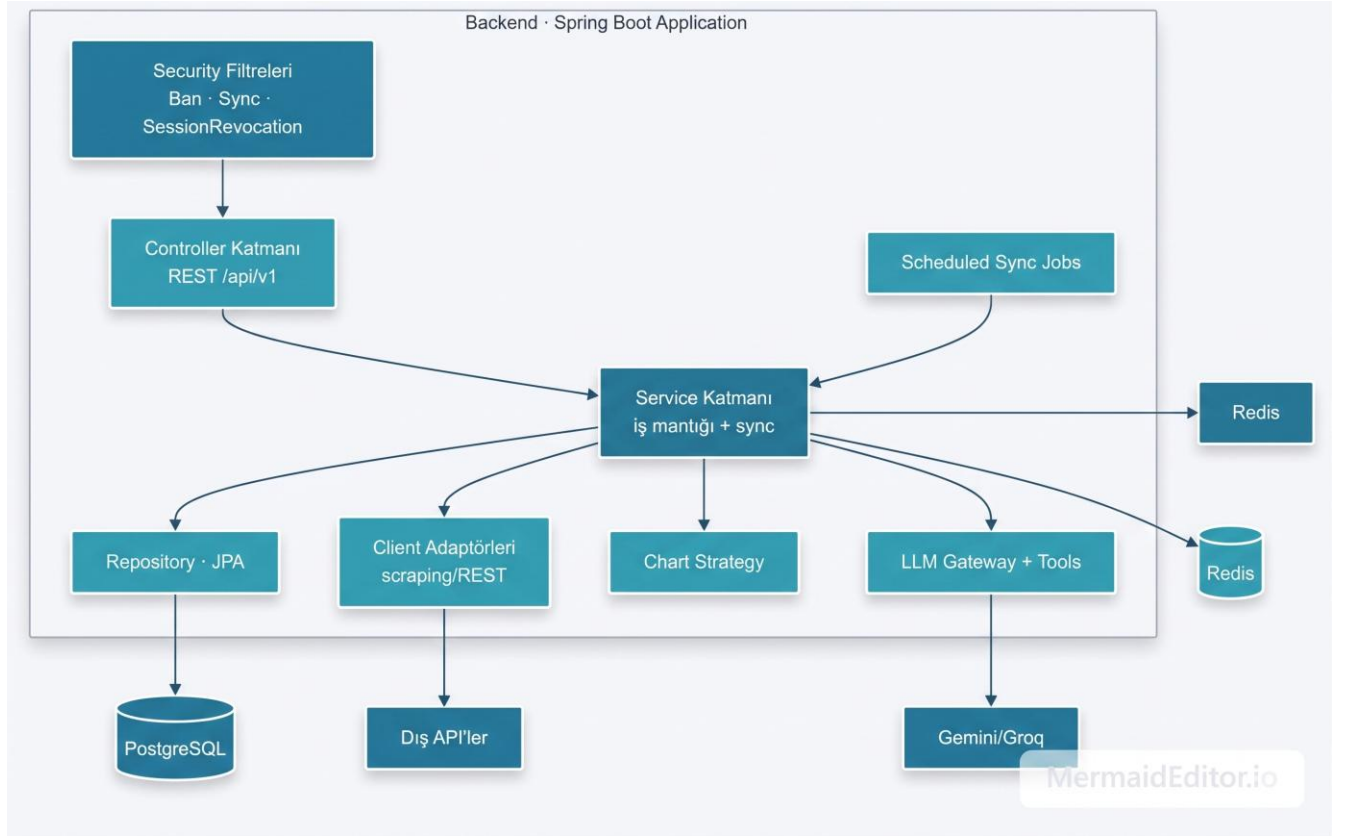


3.3 Bileşen (C4 — Level 3, Backend)

Backend, modüler monolit içinde katmanlı bileşenlerden oluşur (paket: com.otuzikibit.finance_portal):

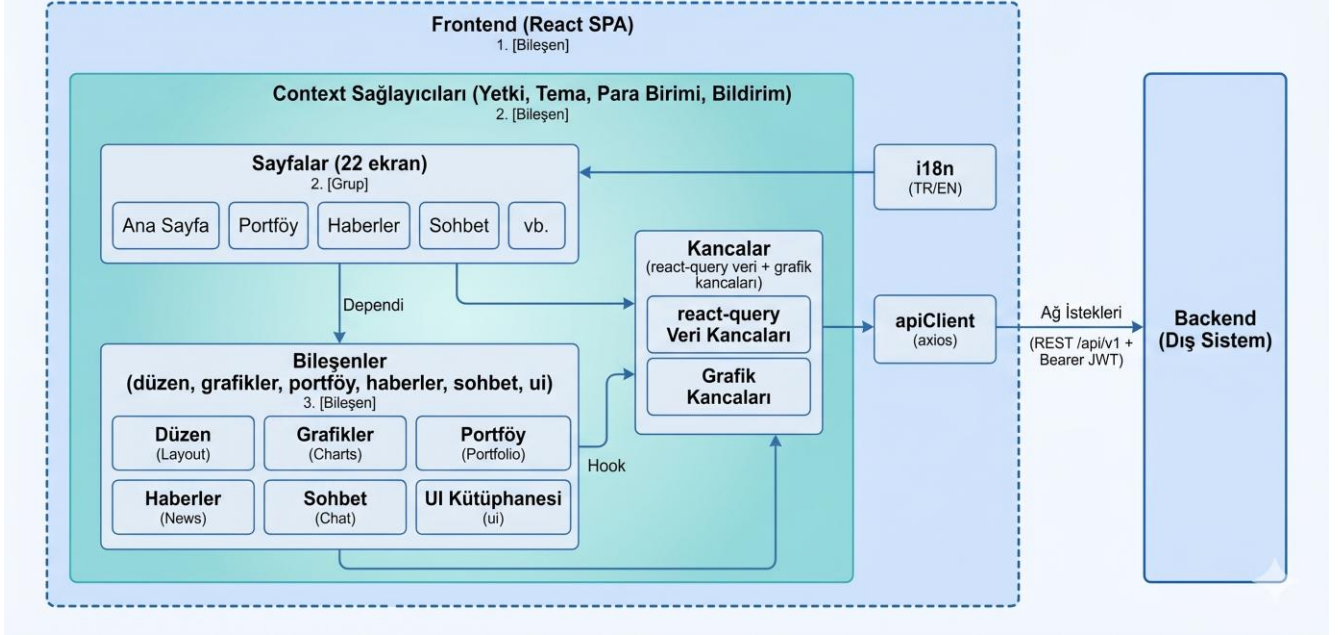
Katman / Bileşen	Paket(ler)	Sorumluluk
Sunum (Controller)	controller/{market,finance,user,alarm,chat}, domains/*/controller	REST uçları, istek/yanıt eşleme
Uygulama / Servis	service/{auth,user,alarm,mail,messaging,market,chat}, domains/*/service	İş mantığı, senkronizasyon, orkestrasyon
Alan Modeli	model/{entity,dto,enums}	Kalıcı varlıklar, DTO'lar, enum'lar
Kalıcılık	repository	Spring Data JPA repository'leri
Entegrasyon (Client)	client/{binance,yahoo}, domains/*/client	Dış API/scraping adaptörleri
Grafik Stratejisi	domains/chart/strategy	Kategoriye göre tarihsel veri kaynağı seçimi
AI Sohbet	service/chat/llm, service/chat/tools	LLM gateway (Gemini/Groq), tool-calling
Güvenlik	security	SecurityConfig + filtreler (ban/sync/oturum)
Çapraz Kesen	config, exception, util	Yapılandırma, hata yönetimi, yardımcılar

— C4 Bileşen Diyagramı (Backend)



(Frontend)

REACT SPA FRONTEND BİLEŞEN DİYAGRAMI



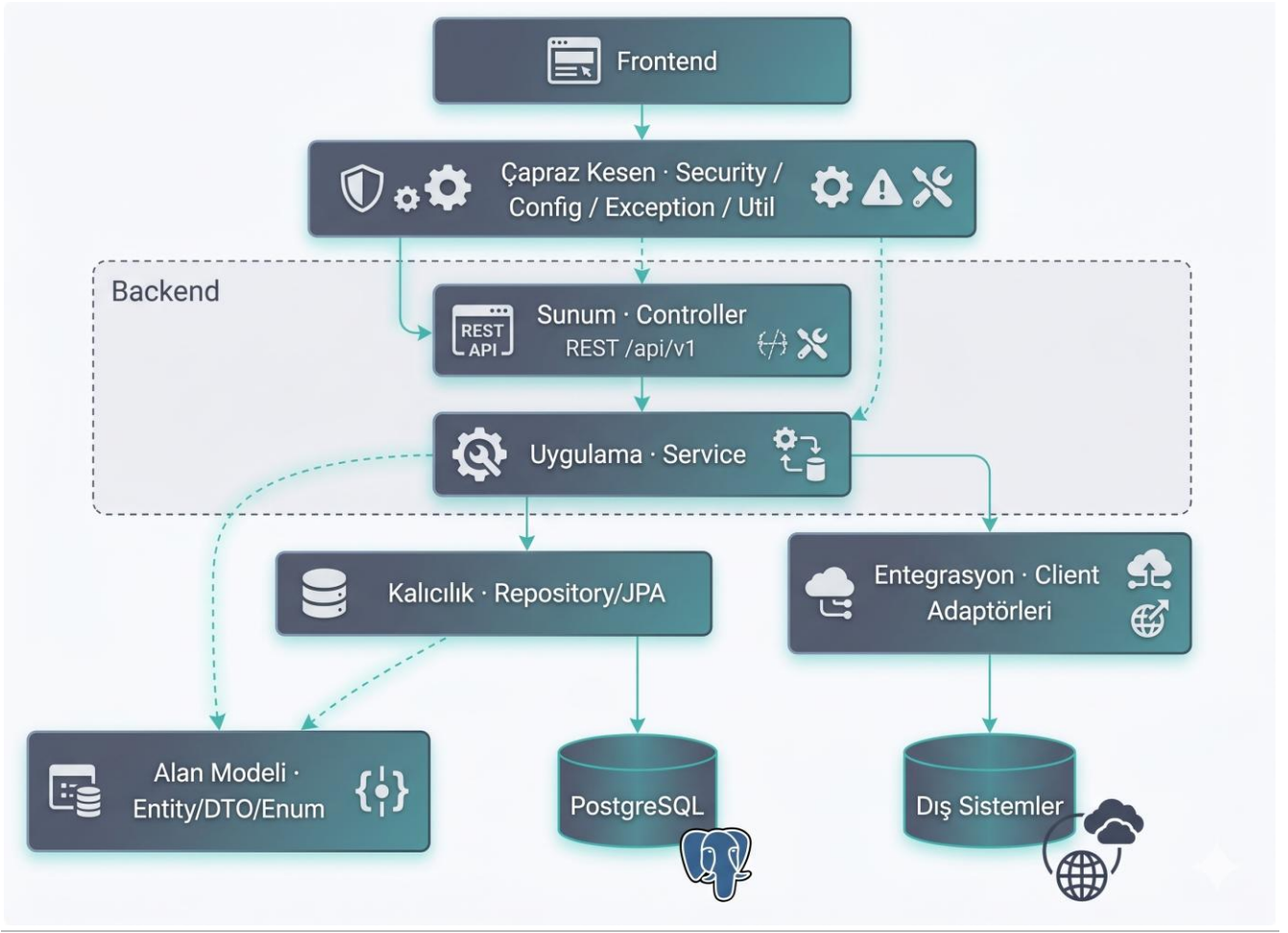
3.4 Clean Architecture Katmanları ve Bağımlılık Kuralı

Backend, **package-by-feature (19 domain) + katmanlı** bir yapı izler (saf hexagonal/port-adapter değil). Bağımlılık tek yönlüdür: dış katmanlar iç katmanlara bağımlıdır, tersi olmaz.

Katman	İçerik	Bağımlı olduğu
Sunum	Controller'lar	Servis
Uygulama/Servis	İş mantığı, senkronizasyon, orkestrasyon	Alan modeli, Repository, Client
Entegrasyon	Client adaptörleri (dış API izolasyonu)	— (dış sistemler)
Kalıcılık	Repository (JPA)	Alan modeli
Alan Modeli	Entity / DTO / enum	— (en iç)

Bağımlılık kuralı: Controller → Service → (Repository | Client | Domain). Dış API'lerin ayrıntısı client/ adaptörlerinde izole edilir; üst katmanlar yalnızca servis arayüzlerini bilir. Güvenlik filtreleri ve yapılandırma **çapraz kesen** olarak tüm isteklere uygulanır.

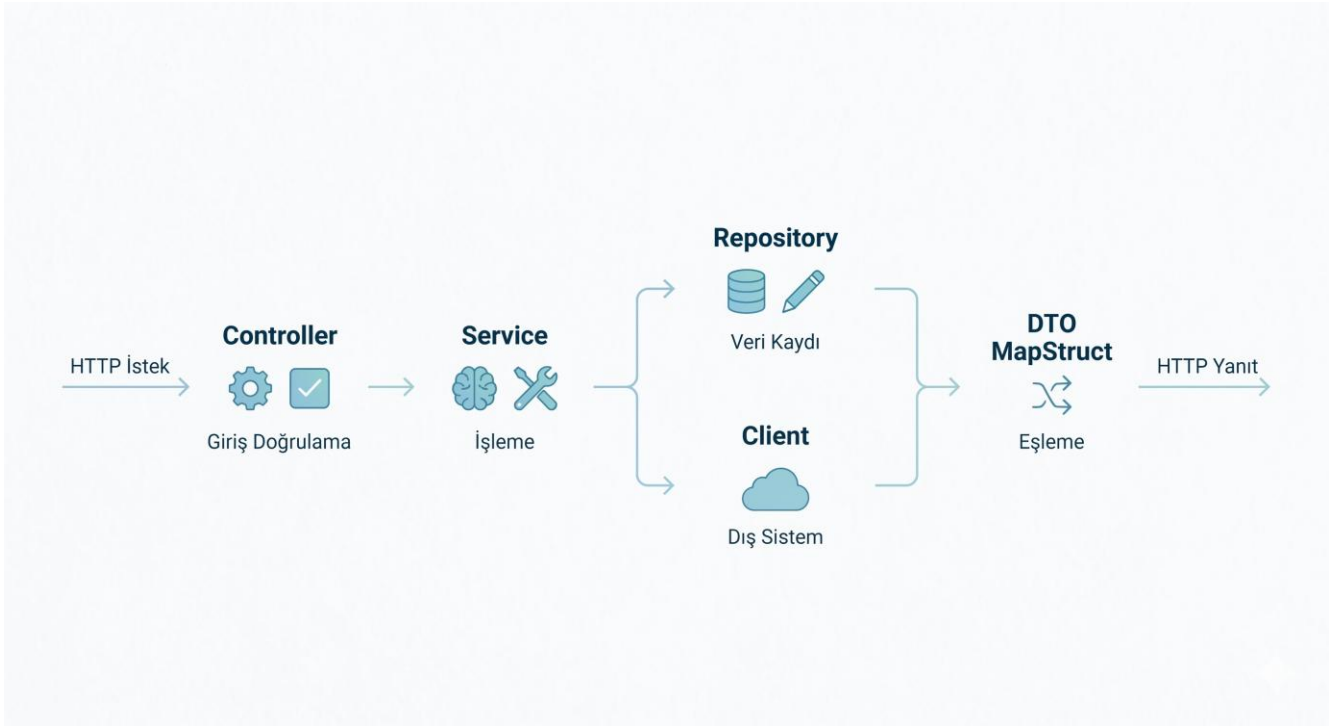
— Katman & Bağımlılık Diyagramı



4. Backend Tasarımı

4.1 Katmanlı Yapı ve İstek Akışı

Her istek katmanlar arasında tek yönlü akar:



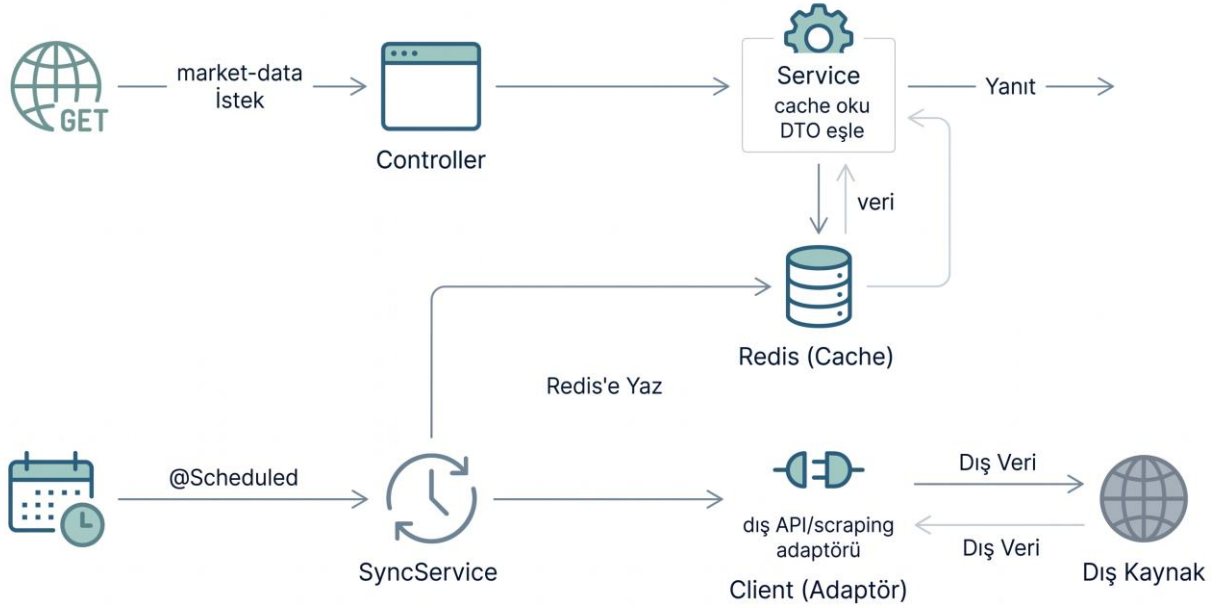
- **Controller:** İnce; yalnızca uç tanımı, parametre/DTO doğrulama (@Valid), servis çağırısı.
- **Service:** İş kuralları, senkronizasyon, orkestrasyon, önbellek erişimi.
- **Repository / Client:** Kalıcı veri (JPA) ya da dış kaynak (REST/scraping) erişimi.
- **MapStruct + Lombok:** Entity ↔ DTO eşleme ve boilerplate azaltma.

4.2 Domain Modülleri Tasarımı

Sistem 19 alan modülünden oluşur. İki tipik modül deseni vardır:

Desen	Açıklama	Örnek domain'ler
Piyasa-veri modülü	controller + service + client + dto; dış kaynaktan @Scheduled ile çeker, Redis'e yazar, controller cache'ten okur	currency, crypto, commodity, stock, fund, eurobond, viop, ipo, economy...
Kullanıcı-veri modülü	controller + service + repository + dto/entity; CRUD, kullanıcıya özel, PostgreSQL	portfolio, watchlist, alarm, simulation, saved chart, chat

Tipik piyasa-veri modülü anatomisi:



4.3 Dış Veri Entegrasyonu ve Symbol Routing

Her dış kaynak, ayrı bir **client adaptörüyle** izole edilir (ör. TcmbIntegrationClient, CoinGeckoClient, TefasFundClient, BistStockClient, HesapkurduIntegrationClient, TruncgilIntegrationClient, ViopScraperClient, YahooQuoteClient, EvdsClient). HTTP istemcileri RestClientConfig'te ortak zaman aşımıyla yapılandırılır; HTML kaynaklar jsoup ile ayrıştırılır.

Symbol Routing (sembol yönlendirme): Aynı varlık farklı kaynaklarda farklı sembolle geçer. İki mekanizma çözer:

1. **Backend YahooSymbolResolver:** Kullanıcı sembolünü kaynak sembolüne normalize eder — FIAT → {KOD}TRY=X, BIST endeks → XU100.IS, kripto → {KOD}-USD, bilinen ETF/=F/=X/.IS → olduğu gibi.
2. **Kategori yönlendirme:** Frontend URL'de ?cat= kategori taşır; gelmezse (UNKNOWN) sembol kalıbından kategori tespit edilir; kategori, doğru **Chart Strategy**'ye ve doğru servise yönlendirir.

— Symbol Routing & Kategori Çözümleme



4.4 Chart Strategy Deseni

Tarihsel veri kaynağı, varlık kategorisine göre **Strategy deseniyle** seçilir: ChartDataStrategy arayüzü + 12 uygulama (CurrencyChartStrategy, CryptoChartStrategy, TurkishStockChartStrategy, FundChartStrategy, EurobondChartStrategy, ViopChartStrategy, YahooDefaultChartStrategy ...).

MarketChartService.getHistoricalDataWithEvdsFallback(...) uygun stratejiyi seçer; strateji yoksa Yahoo varsayılana düşer (ayrıntı SRS §4.4).

4.5 Dayanıklılık (Resilience)

Sistem, **devre kesici (circuit breaker)** yerine "senkronize-et-ve-önbellekten-servis-et" desenini kullanır:

Mekanizma	Açıklama
Periyodik senkronizasyon	@Scheduled job dış kaynaktan çeker, Redis 'e yazar; kullanıcı isteği dış API beklemez
Son-iyi (last-good) snapshot	Kaynak boş/erişilemez ise son başarılı veri sunulur (ör. BİST hafta sonu 48 saat)
Yedek hesaplama	Ör. Türk altını Truncgil bloklanırsa Yahoo ons + USD kurundan hesaplanır
LLM failover	Birincil (Gemini) başarısızsa yedek (Groq); LlmGateway yönetir
Zaman aşımı	Dış HTTP çağrılarında connect/read timeout (RestClientConfig, TranslationClient)

4.6 Hata Yönetimi

Merkezi @RestControllerAdvice (GlobalExceptionHandler) tüm hataları tutarlı bir **ErrorResponse** (timestamp, status, error, message, path, validationErrors) sözleşmesine dönüştürür:

İstisna	HTTP	Not
ResourceNotFoundException	404	Kayıt bulunamadı
IllegalArgumentException / InsufficientBalanceException	400	Hatalı istek
SecurityException	403	Sahiplik/IDOR koruması ihlali
DataIntegrityViolationException	409	Benzersizlik/FK kısıtı çakışması
MethodArgumentNotValidException	400	@Valid alan doğrulama hataları (alan-mesaj haritası)
LlmException	503	LLM erişilemez → kullanıcı dostu mesaj
ClientAbortException / async	—	İstemci bağlantıyı kesti; sessiz (DEBUG iz)
Exception (genel)	500	Son savunma hattı

4.7 Önbellekleme ve Zamanlanmış İşler

Önbellek: RedisConfig → RedisTemplate (String anahtar + GenericJackson2Json değer). TTL ve anahtar ad alanları **servis düzeyinde** belirlenir (global CacheManager yerine ince denetim): ör. cache:*, evds:*, market:*, cache:news:content:en:*

Zamanlanmış işler: Düz Spring @Scheduled (dağıtık kilit/ShedLock **yok**). Örnek tetiklenmeler:

İş	Periyot
Piyasa sync (kripto/emtia/fon/future...)	uygulama açılışı + 5 dk / saatlik
EVDS tarihçe (döviz/ekonomi/tahvil)	açılış + günlük cron (ör. 16:30/16:45)
Fiyat alarmı değerlendirme	2 dk (app.alarm.eval-rate-ms)
Ban süresi dolumu	periyodik (BanExpiryJob)

Tasarım notu: ShedLock olmadığından, backend birden çok replika ile çalıştırılırsa zamanlanmış işler her replikada çalışır. Mevcut dağıtımda backend tek aktif örnek olacak şekilde tasarlanmıştır (bkz. §9.4); çok-örnekli ölçeklemede dağıtık kilit eklenmelidir (bkz. §13 Riskler).

4.8 Konfigürasyon ve Secret Yönetimi

Profiller (5): dev (SQL log/DEBUG), local (API key'ler — gitignored), docker (konteyner DNS), prod (sıkılaştırma: log WARN, actuator kısıtlı), test (H2 in-memory). Override sırası: profil dosyası > application.yaml > kod default'u.

Config sınıfı	Sorumluluk
RedisConfig	RedisTemplate serileştirme
KafkaConfig	Producer/consumer serileştirme
RestClientConfig	Dış HTTP istemcileri + zaman aşımı
JacksonConfig	JSON (JSR-310 tarih/saat) ayarları
OpenApiConfig	Swagger/OpenAPI tanımı
LlmConfig	Gemini/Groq istemci yapılandırması

Secret yönetimi: Tüm gizli değerler **ortam değişkeni** ile gelir (DB/Keycloak/SMTP/API key'ler). Yerelde application-local.yaml (gitignored); üretimde **Kubernetes Secret** (k8s/secrets/10-secrets). Kodda/git'te sır tutulmaz.

4.9 OOP Prensipleri ve Tasarım Desenleri

Tasarım, nesne yönelimli prensiplere ve yerleşik tasarım desenlerine dayanır:

Prensip / Desen	Uygulama
Tek Sorumluluk (SRP) + Katmanlı mimari	Controller → Service → Repository/Client; her katman tek sorumluluk taşır
Bağımlılık Enjeksiyonu (DI)	Spring IoC; constructor injection (@RequiredArgsConstructor)
Strategy	ChartDataStrategy + 12 uygulama (kategoriye göre tarihsel veri kaynağı seçimi)
Adapter	client/ adaptörleri — her dış API/scraping kaynağını tek arayüze indirger
Repository	Spring Data JPA ile kalıcılık soyutlaması
DTO + Mapper	DTO ile katman izolasyonu; MapStruct ile entity ↔ DTO eşleme
Factory / SPI	Keycloak BanCheckAuthenticatorFactory (Service Provider Interface)
Açık/Kapalı (OCP)	Yeni domain veya yeni grafik stratejisi, mevcut kodu değiştirmeden eklenir
Kapsülleme (Encapsulation)	İş kuralları servis katmanında; controller ince tutulur

5. Frontend Tasarımı

5.1 Uygulama Mimarisi

React 19 + Vite tabanlı SPA, sorumluluklara göre klasörlenir:

Klasör	Sorumluluk
pages/	Sayfa bileşenleri (22 sayfa)

components/	Yeniden kullanılabilir bileşenler (layout, charts, portfolio, news, chat, ui...)
context/	Global durum sağlayıcıları (Auth, Theme, Currency, Notification)
hooks/	Veri/iş mantığı hook'ları (react-query tabanlı)
config/	apiClient (axios), grafik overlay yapılandırması
constants/	Tema token'ları, varlık/kategori sabitleri
i18n/	Dil yapılandırması ve çeviri dosyaları
utils/	Token yönetimi, Keycloak yardımcıları, biçimlendiriciler

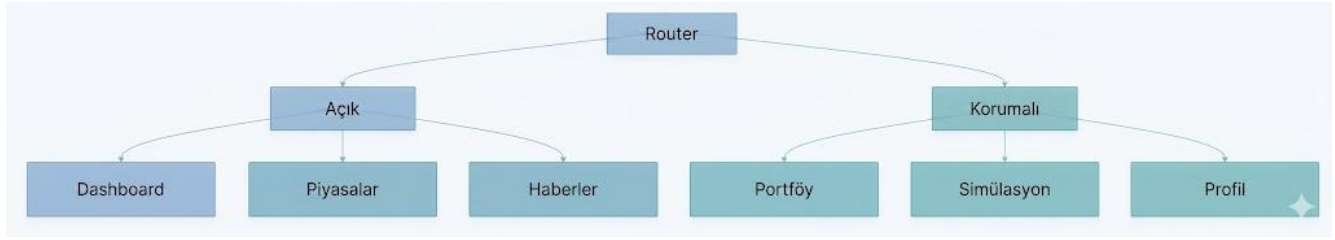
Sağlayıcı hiyerarşisi: AuthProvider → NotificationProvider → CurrencyProvider → Router. **Global bileşenler:** NavBar, GlobalTicker (piyasa şeridi), Footer, ChatWidget (AI sohbet), ScrollToTopButton.

5.2 Yönlendirme ve Sayfa Haritası

Yönlendirme react-router-dom ile; korumalı sayfalar ProtectedRoute sarmalayıcısıyla (giriş yoksa erişim engellenir).

Erişim	Route	Sayfa
Herkese açık	/	Dashboard
Herkese açık	/markets/:category	MarketPage
Herkese açık	/markets/:category/list	MarketListPage
Herkese açık	/chart/:symbol	AssetDetailPage
Herkese açık	/markets/bank-currencies	BankCurrenciesPage
Herkese açık	/markets/turkish-gold	TurkishGoldPage
Herkese açık	/markets/live	LiveMarketPage
Herkese açık	/news, /news/detail	NewsPage, NewsDetailPage
Herkese açık	/economic-calendar	EconomicCalendarPage
Herkese açık	/interest	InterestPage
Callback	/auth/callback, /callback	Keycloak dönüş işleme
Korumalı	/profile	ProfilePage
Korumalı	/portfolio	PortfolioPage
Korumalı	/watchlist	WatchlistPage
Korumalı	/simulation	SimulationPage
Korumalı	/what-if	WhatIfPage
Korumalı	/alarms	AlarmsPage
Korumalı	/saved-charts	SavedChartsPage
Korumalı	/preferences	PreferencesPage
Korumalı	/admin	AdminPage

— Sayfa / Route Haritası



5.3 Durum Yönetimi ve Veri Çekme Katmanı

Katman	Teknoloji / Bileşen
Sunucu durumu	@tanstack/react-query — ön bellek, yeniden getirme, yükleme/hata durumları
HTTP istemcisi	axios (config/apiClient.js) — baseURL .../api/v1, 10 sn timeout
İstek interceptor	Her isteğe Bearer <token> ekler (tokenManager/localStorage)
Yanıt interceptor	response.data döndürür; 403+ban → token temizle + /?banned=true (Dashboard ban pop-up); 401 → token temizle + /
İstemci durumu	Context: Auth (oturum/kullanıcı), Theme (tema), Currency (görüntü para birimi), Notification (uyarılar)
Özel hook'lar	useMarketData, useAssetDetails, useChartData, useNewsData, useTickerData, useUserPreferences, useDashboardData/Layout, useLiveMarketData, useComparisonData ...

5.4 Grafik Bileşenleri Mimarisi

Grafikler, varlık türüne göre ayrı bileşen aileleriyle ele alınır:

Bileşen / Kütüphane	Kullanım
TradingChart (klinecharts)	Mum/teknik gösterge grafikleri (genel varlıklar)
FundTradingChart	Fon tarihçesi grafiği
ViopTradingChart	ViOP kontrat grafiği
lightweight-charts	Hafif fiyat/sparkline grafikleri
recharts	Karşılaştırma, getiri eğrisi, dashboard görselleri
config/customOverlays	Grafik üzerine özel çizim katmanları (kaydedilebilir)

Her grafik ailesi kendi hooks/charts/* veri hook'undan beslenir (useChartData, useFundChartData, useViopChartData).

5.5 Tema Sistemi

Tema, ThemeContext ile yönetilir; renkler **Tailwind CSS** üzerinden **anlamsal token'larla** (bg, text, surface, primary, buy, sell ...) tanımlıdır (constants/theme.js). Tema değişimi tüm arayüze anında yansır ve kullanıcı tercihi kalıcıdır. Token tabanlı yaklaşım, bileşenlerin sabit renk yerine anlamsal değişken kullanmasını sağlar.

5.6 Uluslararasılaştırma (i18n) Mimarisi

Öge	Tasarım
Kütüphane	i18next + react-i18next

Dil tespiti	i18next-browser-languagedetector
Diller	TR / EN — çalışma zamanında değiştirilebilir
Çeviri dosyaları	src/i18n/locales/{tr,en}/*.json — 21 namespace (auth, dashboard, portfolio, charts, news, admin, common, navbar...)
Yapılandırma	src/i18n/index.js
Biçimlendirme	Locale-duyarlı sayı/tarih (Intl / date-fns)

6. Veri Tasarımı

Kalıcı veri **PostgreSQL** üzerinde, şema **Flyway** (V1–V21) ile sürümlenir. Tablolar **normalleştirilmiştir** (3NF): tekrar eden veri yoktur, ilişkiler yabancı anahtarlarla (FK) kurulur. Piyasa verileri kalıcı tutulmaz; **Redis**'te geçici önbelklenir (§6.5).

6.1 Mantıksal Veri Modeli (ER)

Merkezde **users**; tüm kişisel veriler `user_id` ile bağlı ve ON DELETE CASCADE.

— Mantıksal Varlık-İlişki (ER) Diyagramı



6.2 Veritabanı Şeması

users

Kolon	Tip	Kısıt
id	UUID	PK
username	VARCHAR(50)	NOT NULL, UNIQUE
email	VARCHAR(100)	NOT NULL, UNIQUE
password	VARCHAR(255)	(legacy; kimlik Keycloak'ta)
role	VARCHAR(50)	USER / ADMIN
banned_until	TIMESTAMP	geçici ban bitişi
ban_permanent	BOOLEAN	DEFAULT false
ticker_scope	VARCHAR(20)	DEFAULT 'ALL_PAGES'
email_notifications_enabled	BOOLEAN	DEFAULT true
session_invalidated_at	TIMESTAMPTZ	force-logout zamanı
created_at	TIMESTAMP	NOT NULL

portfolios / portfolio_items

Tablo	Kolonlar (özet)	Kısıt / İndeks
portfolios	id PK, user_id FK, name VARCHAR(120), created_at	FK→users CASCADE; idx(user_id)
portfolio_items	id PK, user_id FK, portfolio_id FK, symbol, asset_type, quantity NUMERIC(19,8), average_price NUMERIC(19,8), contract_size NUMERIC(19,6) DEFAULT 1	FK→users/portfolios CASCADE; idx(portfolio_id)

transactions

Kolon	Tip	Kısıt
id	UUID	PK
user_id	UUID	FK→users CASCADE
symbol / asset_type	VARCHAR(50)	NOT NULL
side	VARCHAR(4)	CHECK (BUY/SELL)
quantity	NUMERIC(19,6)	NOT NULL
price	NUMERIC(19,4)	NOT NULL
executed_at	TIMESTAMP	NOT NULL
notes	TEXT	—

İndeksler: (user_id, symbol), (user_id, executed_at).

Diğer kullanıcı tabloları

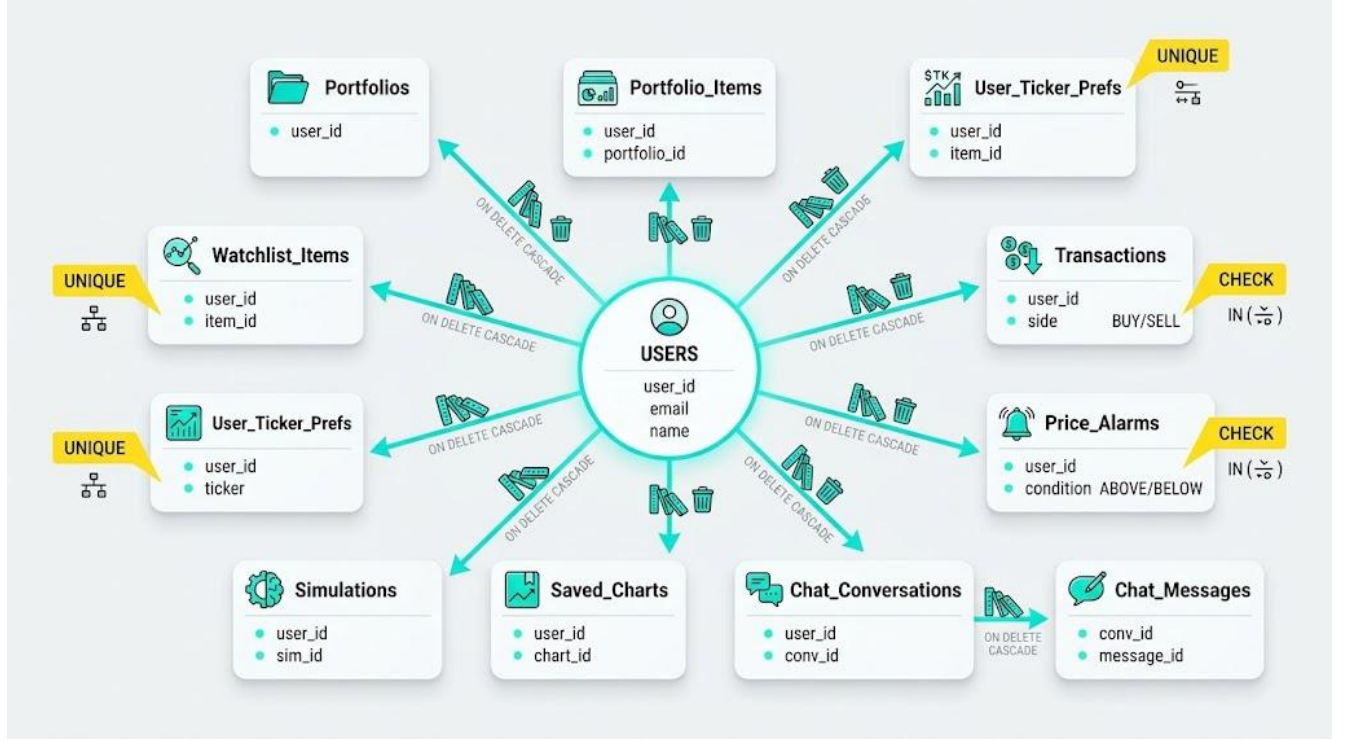
Tablo	Kolonlar (özet)	Kısıt / İndeks
watchlist_items	id, user_id, symbol, asset_type, added_at	UNIQUE(user_id,symbol,asset_type); idx(user_id)
simulations	id, user_id, symbol, asset_type, investment_date DATE, amount_try NUMERIC(19,4), notes, created_at	FK CASCADE; idx(user_id)
user_ticker_prefs	id, user_id, symbol, asset_type, display_order INT	UNIQUE(user_id,symbol,asset_type); idx(user_id)

saved_charts	id, user_id, symbol VARCHAR(64), asset_category, name, payload TEXT, created_at, updated_at	FK CASCADE; idx(user_id, created_at DESC)
price_alarms	id, user_id, symbol, asset_type, condition (ABOVE/BELOW), threshold NUMERIC(24,8), frequency (ONCE/CONTINUOUS), note, active, last_triggered_at, trigger_count	FK CASCADE; idx(active), idx(user_id)
chat_conversations	id, user_id, title, created_at, updated_at	FK CASCADE; idx(user_id), idx(user_id, updated_at DESC)
chat_messages	id, conversation_id, role (USER/ASSISTANT/SYSTEM/TOOL), content, tool_calls (JSON), tool_name, model_used, created_at	FK→chat_conversations CASCADE; idx(conversation_id, created_at)

6.3 Veri Bütünlüğü Kuralları

Kural Türü	Uygulama
Birincil anahtar	Tüm tablolarda UUID PK
Yabancı anahtar	Tüm kullanıcı verileri user_id→users(id) ON DELETE CASCADE (kullanıcı silinince tüm verisi silinir — KVKK ile uyumlu)
Benzersizlik	users(username), users(email); watchlist_items ve user_ticker_prefs'te (user_id, symbol, asset_type)
CHECK / enum	transactions.side ∈ {BUY,SELL}; price_alarms.condition ∈ {ABOVE,BELOW}, frequency ∈ {ONCE,CONTINUOUS}
Hassasiyet	Parasal/oran alanları NUMERIC (ondalık kayıp yok): miktar (19,8/19,6), fiyat (19,4), eşik (24,8)
Uygulama düzeyi	Sahiplik kontrolü (IDOR): başka kullanıcının kaydına erişim SecurityException→403 (§4.6)

— Bütünlük / Cascade Görseli Gemini prompt'u:



6.4 Şema Versiyonlama (Migration) ve Tohum (Seed)

Şema değişiklikleri **Flyway** ile sürümlenir (src/main/resources/db/migration/V*.sql):

Sürüm	Özet
V1	İlk şema (users + eski accounts/transactions)
V2	users.password (legacy)
V3	portfolio_items
V5, V9	users'a role/banned_until, ban_permanent
V7	accounts ve eski transactions DROP (Account/Transaction kapsam dışı)
V8	watchlist_items
V10	simulations
V11, V12	yeni transactions + portfolio_items'tan geri-doldurma (seed)
V13	user_ticker_prefs + users.ticker_scope
V14	portfolio_items.contract_size
V15	portfolios + portfolio_items.portfolio_id + "Ana Portföy" seed
V16	saved_charts
V17	users.risk_profile DROP
V18	price_alarms
V19	users.email_notifications_enabled
V20	chat_conversations + chat_messages
V21	users.session_invalidated_at

Diğer tohum verisi: TR tahvil kataloğu gibi statik veriler uygulama kaynaklarından/JSON'dan yüklenir (*Catalog).

6.5 Önbellek (Redis) Tasarımı

RedisTemplate (String anahtar + JSON değer). Anahtarlar ad alanlarına göre düzenlenir; TTL servis düzeyinde belirlenir:

Anahtar Deseni	İçerik	Örnek TTL
cache:* (bank_currencies, futures:v2, economic-calendar...)	Anlık liste verileri	5–60 dk
evds:currency:{KOD}, evds:history:macro:{metric}	EVDS tarihsel seriler	~24 saat
`market:economy:{turkey usa}`		Ekonomi anlık snapshot
cache:news:content:en:{sha1(url)}	EN makale çevirisi	7 gün

Bu tasarım, dış kaynak gecikmelerini kullanıcıdan soyutlar ve §4.5'teki dayanıklılık desenini destekler.

7. Arayüz / API Tasarımı

7.1 REST API Sözleşmesi

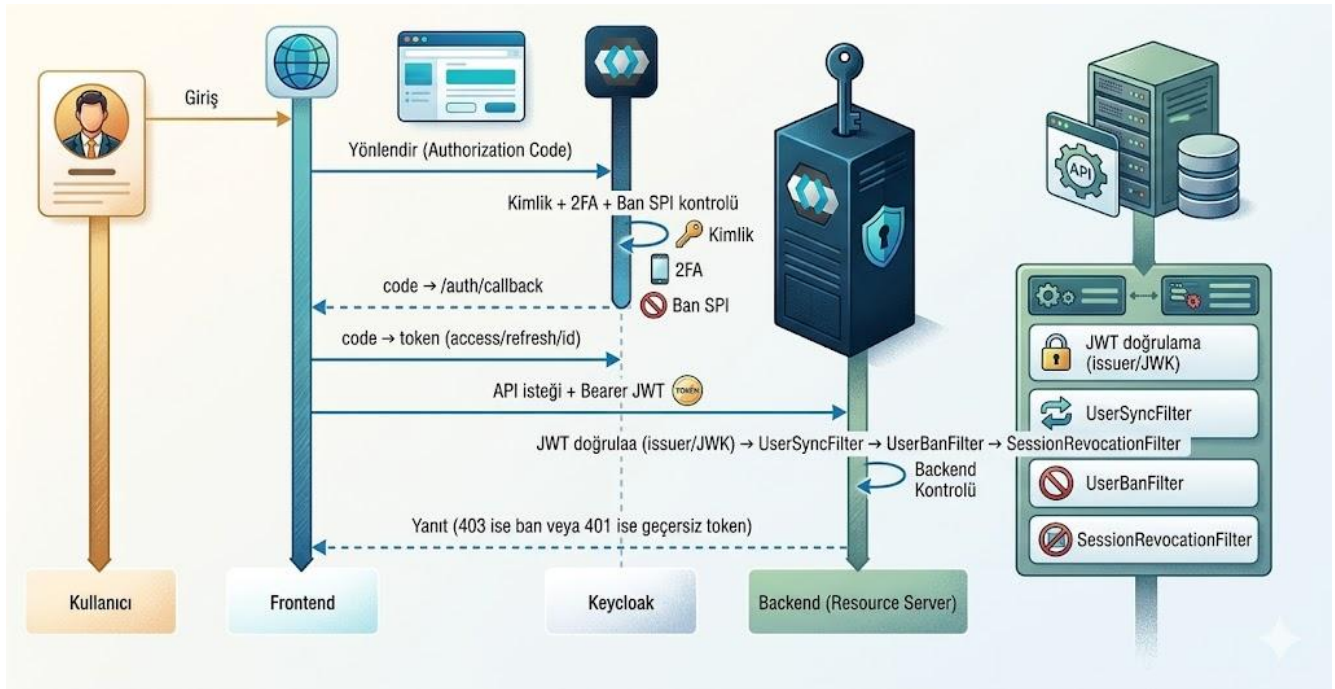
İlke	Tasarım
Kök ön ek	Tüm uçlar /api/v1 (sürümleme; server.servlet.context-path)
Format	İstek/yanıt JSON (UTF-8)
Kimlik	Korumalı uçlar Authorization: Bearer <JWT> ister
HTTP fiilleri	GET (oku), POST (oluştur), PUT (güncelle), DELETE (sil)
Durum kodları	200/201 başarı · 400 doğrulama · 401 kimlik · 403 yetki/ban · 404 yok · 409 çakışma · 503 LLM · 500 genel
Hata gövdesi	Tek tip ErrorResponse (timestamp, status, error, message, path, validationErrors) — §4.6
Sayfalama	Spring Pageable (page, size) + Page<> yanıtı (ör. /admin/users, /portfolio/transactions)
Dokümantasyon	OpenAPI / Swagger UI (/swagger-ui, /v3/api-docs)

Uç kataloğunun tamamı SRS §4'teki modül tablolarındadır; bu doküman sözleşme ilkelerini ve akışları tanımlar.

7.2 Kimlik Doğrulama Akışı (OAuth2 / OIDC)

Kimlik **Keycloak**'a devredilmiştir; backend bir **OAuth2 Resource Server**'dir (JWT doğrular). Frontend, Authorization Code akışıyla token alır; backend her istekte JWT'yi doğrular, kullanıcıyı yerele senkronize eder ve ban kontrolü yapar.

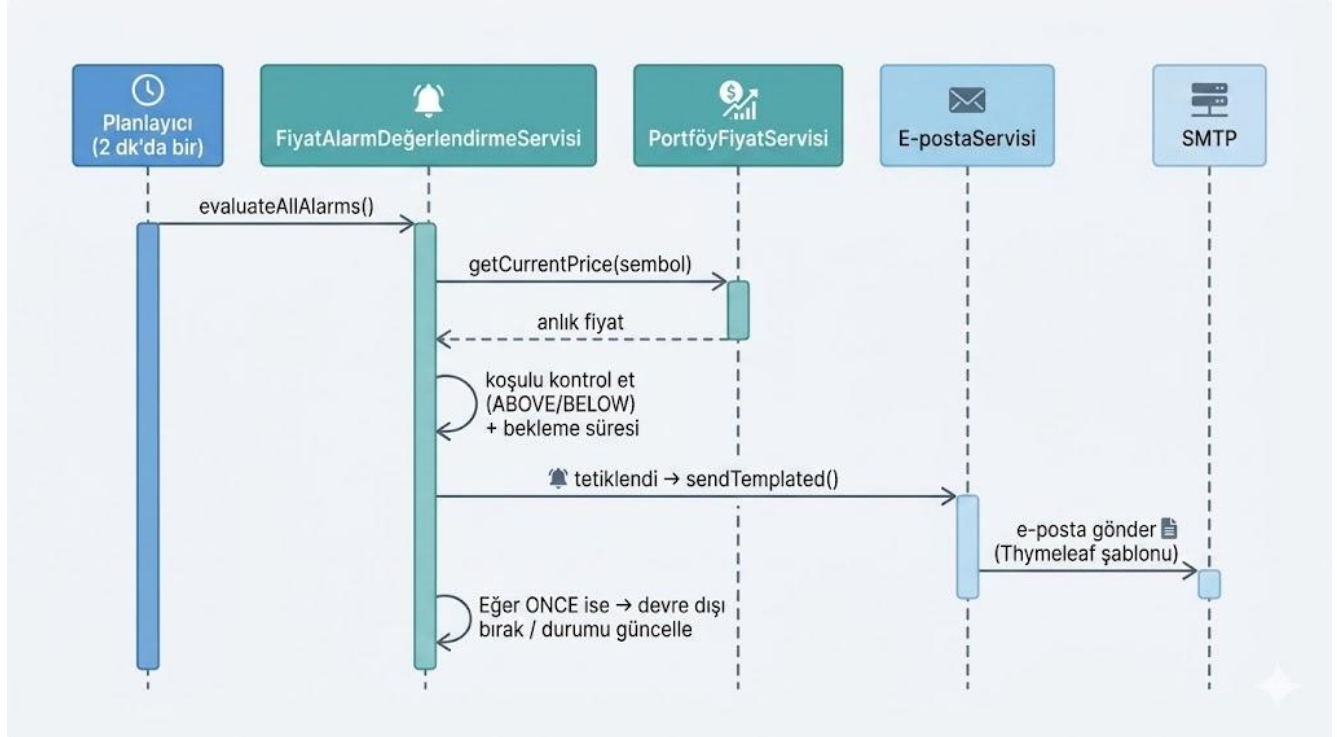
— Kimlik Doğrulama Sıralı Diyagramı



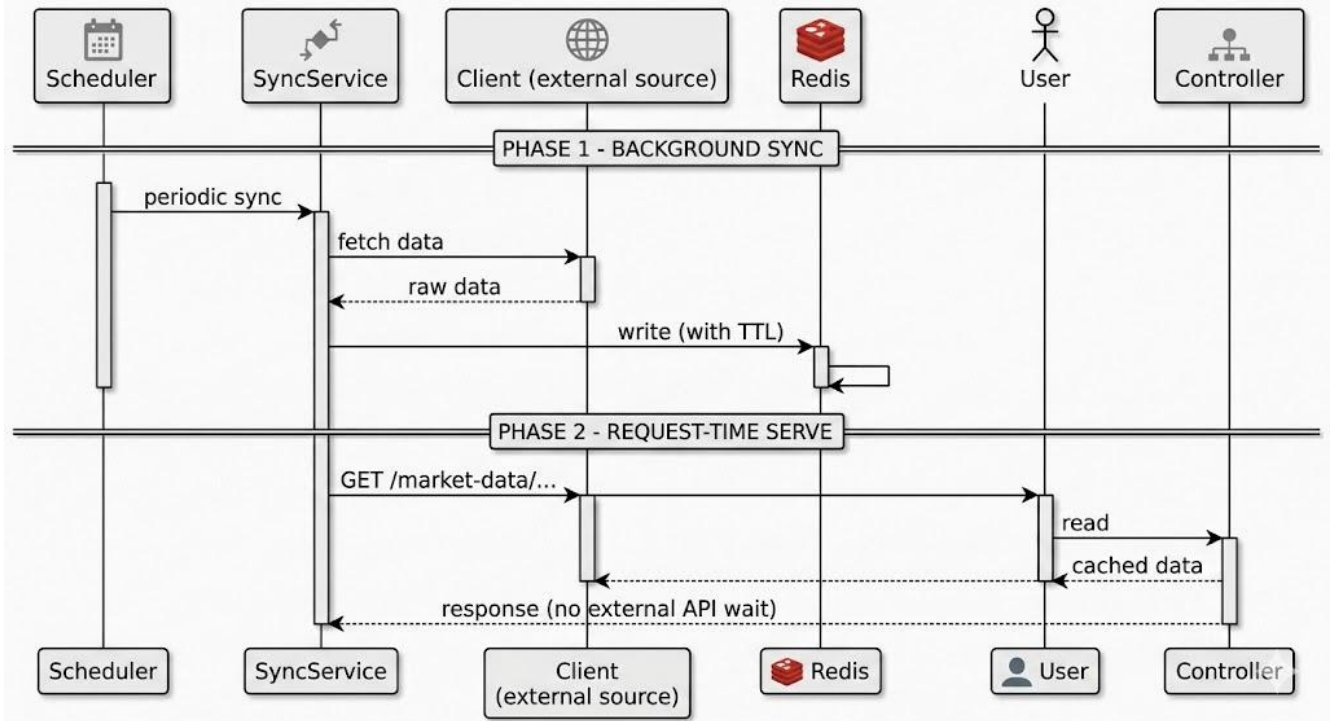
Banlı kullanıcı **Keycloak login akışında 2FA'dan önce** (SPI) engellenir; ayrıca API isteğinde UserBanFilter 403 döner (çift katman).

7.3 Önemli Etkileşim Senaryoları

— Fiyat Alarmı → Bildirim Akışı



— Piyasa Verisi: Senkronizasyon vs Servis



Diğer önemli akışlar: **AI sohbet tool-calling** (SRS §4.11 sıralı diyagramı) ve **portföy değerlendirme** (PortfolioController → PortfolioService → PortfolioPriceService ile anlık fiyat × miktar × sözleşme büyüklüğü → K/Z).

7.4 Dış Entegrasyon Sözleşmeleri ve Degradate

Her dış entegrasyon, hata durumunda **bozulmadan azalan davranış (graceful degradation)** ile tasarlanır:

Entegrasyon	Sözleşme	Hata durumunda
Piyasa kaynakları (TCMB/EVDS/Yahoo/Fintables/iş Yatırım/...)	REST/scrape → Redis	Son-iyi snapshot / yedek hesap / boş liste
Keycloak (OIDC)	JWT doğrulama / Admin API	401/503; admin işlemleri için anlamlı mesaj
Lingva (çeviri)	GET çeviri	null → kaynak dile düş (TR)
Gemini/Groq (LLM)	sohbet tamamlama	failover (primary→fallback); ikisi de düşerse 503
SMTP (e-posta)	alarm bildirimi	log + sessiz (alarm state yine güncellenir)

8. Güvenlik Tasarımı

Güvenlik tasarımı OWASP ASVS (L1–L2 hedef) çerçevesinde ele alınır.

8.1 Kimlik ve Yetki Modeli

Öge	Tasarım
Kimlik sağlayıcı	Keycloak (realm: finance-realm), OIDC

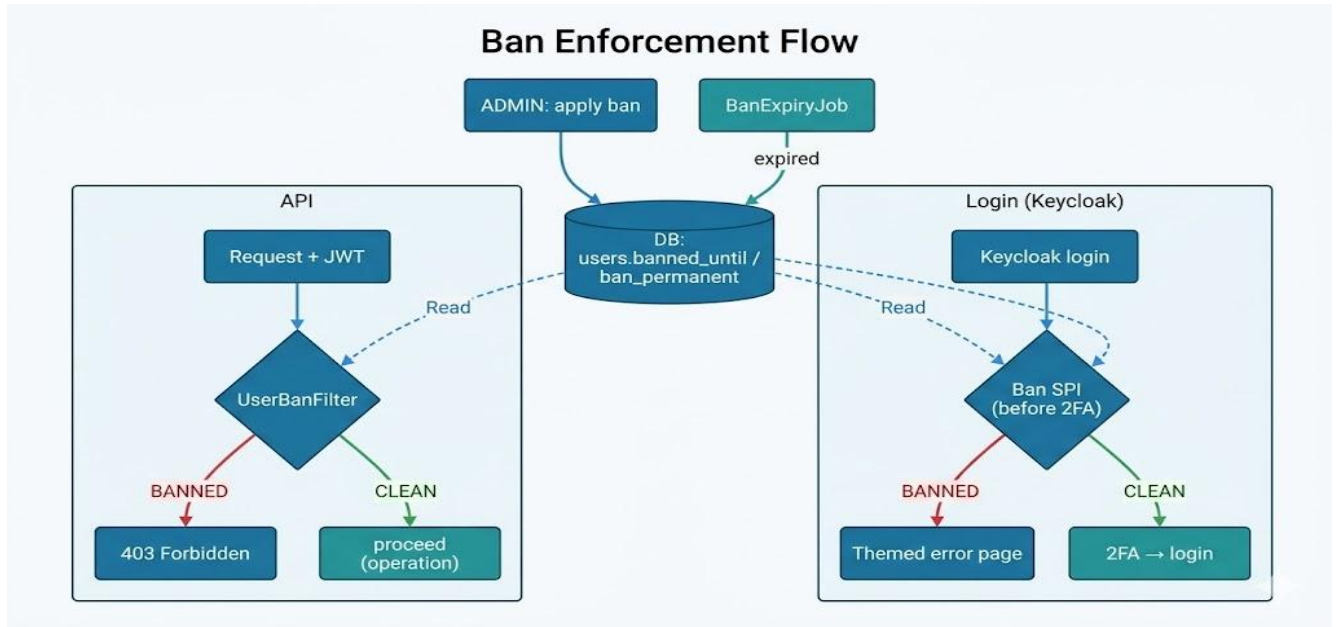
Token	JWT (access/refresh/id); backend OAuth2 Resource Server olarak doğrular (issuer/JWK)
Principal	preferred_username claim'i
Roller	realm_access.roles → ROLE_*; USER / ADMIN
Yetki uygulama	URL bazlı (SecurityConfig) + metot bazlı (@PreAuthorize("hasRole('ADMIN')"))
Parola	Yalnızca Keycloak'ta (backend tutmaz; users.password legacy)
Kullanıcı senkronu	İlk yetkili istekte UserSyncFilter Keycloak kimliğini yerel users tablosuna yansıtır
Beni hatırla	Keycloak realm rememberMe=true — uzun ömürlü oturum seçeneği
Dizin	OpenLDAP altyapıda mevcut (federasyon şu an aktif değil)

8.2 Ban (Hesap Askıya Alma) Mekanizması — Çok Katmanlı

Ban, **dört katmanda** uygulanır; tek nokta atlatılsa bile diğeri yakalar:

Katman	Bileşen	Etki
1. Giriş akışı	Keycloak SPI BanCheckAuthenticator	Parola sonrası, 2FA'dan önce ; users tablosunu (aynı PostgreSQL) native sorguyla okur; banlıysa temalı hata sayfası, OTP ekranı dahi görünmez (DB okunamazsa fail-open)
2. API erişimi	UserBanFilter	Her yetkili istekte ban kontrolü → 403 (PERMANENT/TEMPORARY + bitiş)
3. Oturum iptali	SessionRevocationFilter + users.session_invalidated_at	Force-logout sonrası eski token'lar reddedilir
4. Yönetim	AdminService + KeycloakAdminService + BanExpiryJob	ADMIN ban uygular (DB + Keycloak disable); süre dolan ban otomatik kalkar

— Ban Uygulama Akışı



8.3 CORS, Secret ve Taşıma Güvenliği

Konu	Tasarım
CORS	İzinli origin'ler ortam değişkeniyle (<code>security.cors.allowed-origins</code>); * yasak (<code>allowCredentials=true</code> ile CSRF riski); izinli metod/başlık tanımlı
Secret	Tüm sırlar env / Kubernetes Secret; kodda/git'te yok (§4.8)
Taşıma	Üretimde HTTPS/TLS (<code>cert-manager</code>); HTTP→HTTPS
Oturum	Stateless (JWT); sunucu oturumu yok
Hız sınırı	AI sohbet kullanıcı başına dakikada 20 istek
Girdi doğrulama	DTO'larda <code>@Valid</code> ; doğrulama hatası 400 (alan-mesaj haritası)
Yetkisiz erişim (IDOR)	Kayıt sahipliği kontrolü → <code>SecurityException</code> → 403

8.4 Tehdit Modeli (STRIDE) ve ASVS

STRIDE	Tehdit	Önlem
Spoofing	Sahte kimlik	Keycloak/OIDC, JWT imzası doğrulama
Tampering	Veri/istek değiştirme	TLS, sunucu tarafı doğrulama, JWT imzası
Repudiation	İnkâr	Yapılandırılmış loglama (<code>log4j2 JSON</code> → <code>OpenSearch</code>)
Information Disclosure	Veri sızıntısı	Yetki kontrolü, CORS kısıtı, secret yönetimi, HTTPS
Denial of Service	Aşırı yük	Önbellek (dış API'yi koruma), sohbet hız sınırı, cooldown
Elevation of Privilege	Yetki yükseltme	Rol bazlı yetki (USER/ADMIN), <code>@PreAuthorize</code> , IDOR koruması

ASVS: Kimlik doğrulama (V2), oturum yönetimi (V3), erişim denetimi (V4), girdi doğrulama (V5) ve iletişim güvenliği (V9) maddeleri L1–L2 düzeyinde hedeflenir.

9. Dağıtım (Deployment) Tasarımı

9.1 Konteynerizasyon

İmaj	Dockerfile	İçerik
Backend	<code>finance_portal/Dockerfile</code>	Spring Boot uygulaması + OTel Java Agent
Frontend	<code>finance-frontend/Dockerfile</code>	Vite build → nginx ile statik sunum
Keycloak (özel)	<code>finance_portal/keycloak-image/Dockerfile</code>	Ban SPI (<code>ban-authenticator</code>) + özel temalar gömülü

İmajlar GitHub Actions ile derlenip **Artifact Registry**'e (`eu-west1-docker.pkg.dev`) **git SHA etiketiyle** push edilir.

Orkestrasyon iki seviyede sunulur:

Seviye	Araç	Amaç
--------	------	------

Baz (yerel)	Docker Compose (finance_portal/docker-compose.yml)	Geliştirme/demo; tüm yığını tek makinede ayağa kaldırma
Üretim	Kubernetes (GKE) (k8s/)	Bulutta dağıtım, ölçekleme, kendini iyileştirme

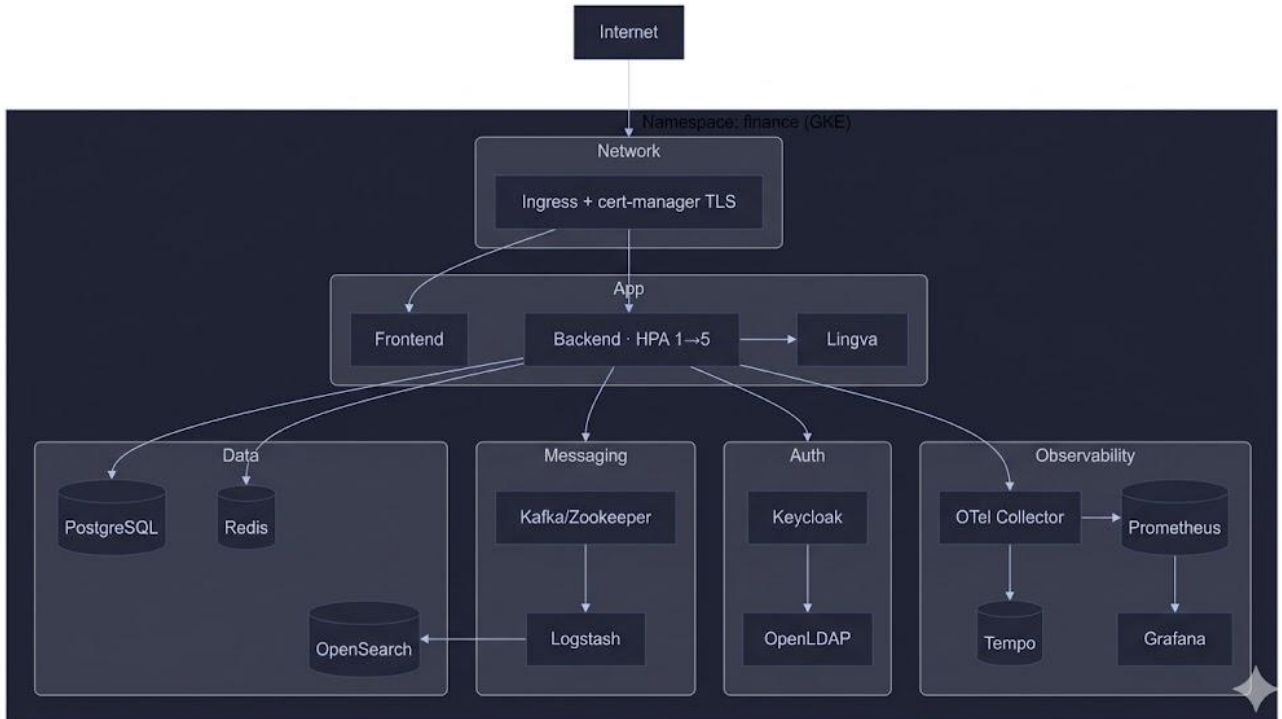
Konteyner teknolojisi **Docker**'dir; Kubernetes bir orkestrasyon katmanıdır. 32Bit gereksinimi "en azından Compose seviyesi" ister — proje Compose'u sağlar ve ek olarak Kubernetes/GKE ile bunu **aşar**.

9.2 Kubernetes Topolojisi

Tüm sistem tek namespace'te (finance) çalışır. Manifestler katmanlı klasör yapısıyla düzenlenir:

Klasör	İçerik
namespace/	Namespace tanımı
secrets/	Kubernetes Secret'lar
configs/	ConfigMap'ler (nginx, logstash, otel, tempo, prometheus, grafana, backend)
data/	PostgreSQL, Redis, OpenSearch
messaging/	Zookeeper, Kafka, Logstash
auth/	OpenLDAP, phpLDAPadmin, Keycloak
app/	Lingva, Backend, Frontend
observability/	OTel Collector, Tempo, Prometheus, Grafana, OpenSearch Dashboards
scaling/	Backend HPA
network/	Ingress, cert-manager, Let's Encrypt issuer

— Kubernetes Dağıtım Topolojisi Gemini prompt'u



9.3 Ağ, Ingress ve TLS

Öge	Tasarım
Giriş	Ingress (nginx) — / → frontend, /api → backend
TLS	cert-manager + Let's Encrypt issuer ile otomatik sertifika
Host	<nginx-lb-ip>.nip.io (demo); HTTPS
Autopilot uyumu	cert-manager için ek RBAC (network/71b-cert-manager-autopilot-rbac)

9.4 Ölçekleme, Depolama ve Secret

Kaynak	Tasarım
Ölçekleme	Backend HPA : CPU %70 / bellek %80 → 1→5 replika; hızlı yukarı (30sn, +2 pod), yavaş aşağı (180sn)
Depolama	Kalıcı bileşenler (PostgreSQL/Redis/OpenSearch) için PVC
Secret	k8s/secrets/ üzerinden env enjeksiyonu

Önemli tasarım notu: Backend HPA ile 5 replikaya çıkabildiğinden ve zamanlanmış işlerde dağıtık kilit (ShedLock) bulunmadığından, ölçeklenmiş durumda @Scheduled senkronizasyon işleri her replikada çalışır (mükerrer dış çağrı; alarmda çift bildirim riski). Önbelik yazımları idempotent olduğundan veri tutarlılığı bozulmaz; ancak çok-replikalı kalıcı kullanım için **dağıtık kilit eklenmelidir** (bkz. §13 Riskler).

9.5 CI/CD Boru Hattı

Aşama	Tetikleyici	Adımlar
CI (ci-cd.yml)	push / PR (main, master)	JDK 21 → mvn verify (derleme + unit + IT + JaCoCo) → kapsam raporu artifact (14 gün) → docker compose config kontrolü
CD (deploy-to-gke.yml)	manuel (workflow_dispatch) veya main'e ilgili-yol push	GCP auth (Service Account JSON key) → gcloud/GKE → backend & frontend imaj build+push (git SHA) → manifest apply → kubectl set image (rolling update) → rollout bekle → liveness smoke test

Yerel tek-komut ayağa kaldırma: scripts/open-cluster.ps1 (sunum/geliştirme için tüm yığını GKE'ye getirir); scripts/close-cluster.ps1 ile kapatılır.

Not: Statik kod analizi (SonarQube) yerelde çalıştırılır (özel repo'da SonarCloud ücretli olduğundan CI'da koşmaz).

10. Gözlemlenebilirlik Tasarımı

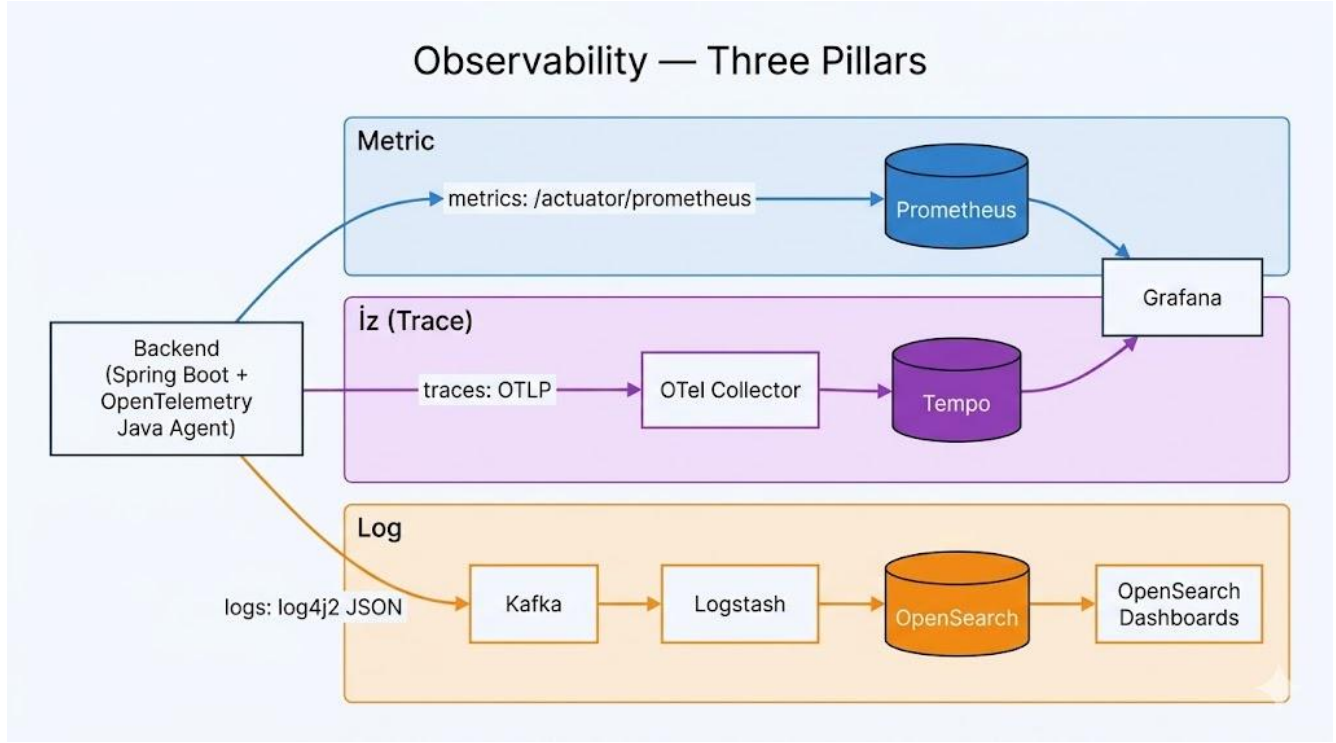
Gözlemlenebilirlik, **OpenTelemetry** temelli üç sütun (metrik, iz, log) üzerine kuruludur.

10.1 Üç Sütun Mimarisi

Sütun	Toplama	Depolama	Görselleştirme
Metrik	Micrometer → /actuator/prometheus	Prometheus	Grafana
İz (Trace)	OpenTelemetry Java Agent	OTel Collector → Tempo	Grafana

	(-javaagent) → OTLP		
Log	log4j2 JSON layout → Kafka	Kafka → Logstash → OpenSearch	OpenSearch Dashboards

— Gözlemlenebilirlik (3-Sütun) Veri Akışı



10.2 Bileşenler ve Kararlar

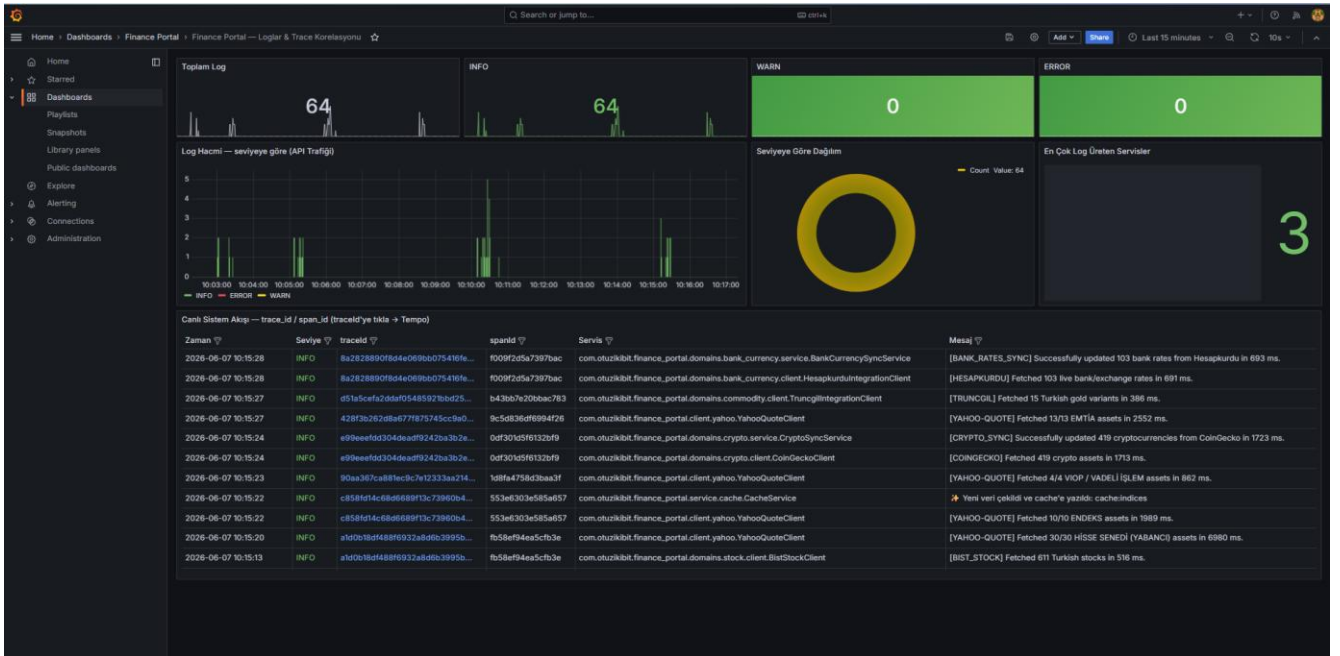
Bileşen	Rol / Karar
Micrometer (Prometheus registry)	/actuator/prometheus ucunu yayınlar; Prometheus doğrudan scrape eder (Collector çökse bile JVM/HTTP metrikleri görünür)
OTel Java Agent	Bytecode instrumentation ile otomatik iz; in-code tracer yok (çakışma önlemek için); örnekleme OTEL_TRACES_SAMPLER env'i ile
OTel Collector	Telemetriyi toplar/iletir (Tempo, Prometheus)
Tempo / Prometheus / Grafana	İz / metrik depolama + birleşik panolar
Kafka → Logstash → OpenSearch	Yapılandırılmış log boru hattı; JSON alanları: timestamp, level, serviceName, message, exception (log4j2-spring.xml)
Actuator	health (liveness/readiness), info, prometheus, metrics uçları

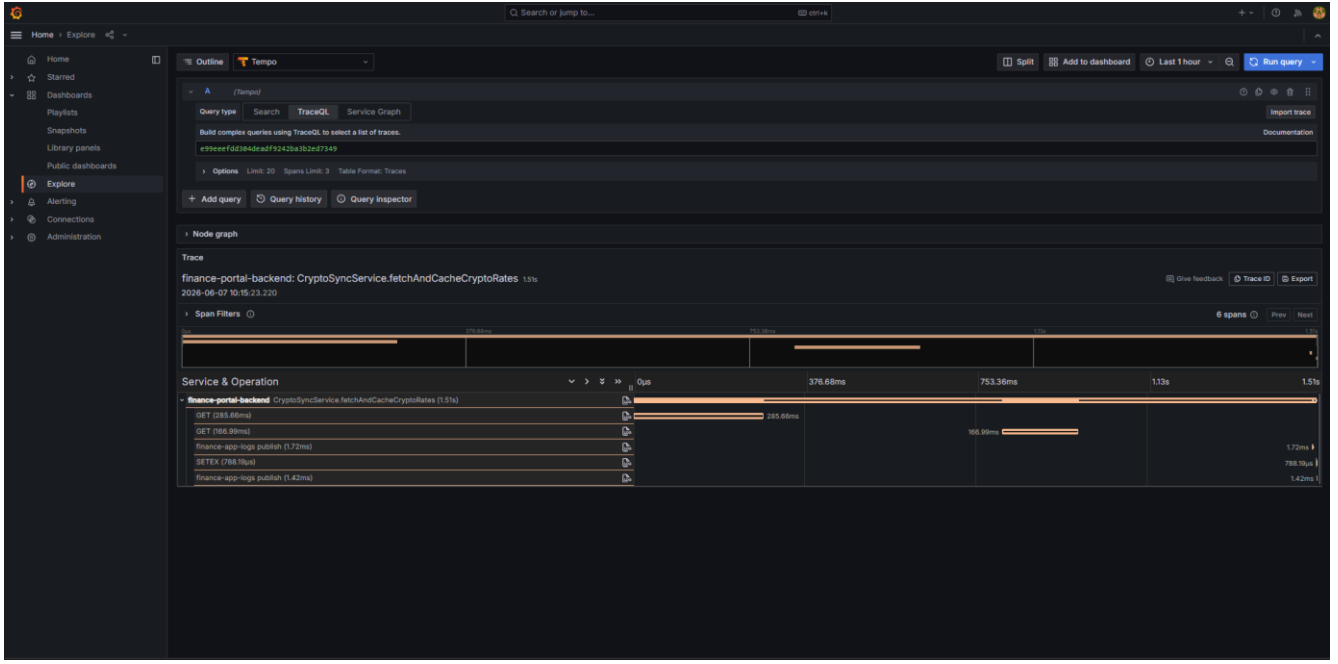
10.3 Pano ve Uyarı Stratejisi

Pano / Metrik	İçerik
HTTP yanıt süreleri	http.server.requests histogramı → P50/P95/P99 (SLO bucket'ları: 50ms–2s)
Verimlilik / hata	İstek/sn, hata oranı (durum kodu bazlı)

JVM / sistem	Bellek, GC, thread, CPU
İz korelasyonu	Yavaş istekler için Tempo'da uçtan uca trace
Loglar	OpenSearch Dashboards'ta arama/filtre (seviye, servis, korelasyon)

Örnek Grafana panoları (32Bit isteri \$6 ile uyumlu):





Sağlık & kurtarma: Kubernetes liveness/readiness probe'ları actuator health uçlarını kullanır; başarısız pod otomatik yeniden başlatılır. Performans hedefi (P95 < 2 sn) Grafana'da histogram_quantile sorgusuyla ve k6 yük testiyle doğrulanır (SRS §7.2).

11. Çapraz Kesen Konular (Cross-Cutting Concerns)

Aşağıdaki konular tek bir modüle ait olmayıp tüm sistemde tutarlı biçimde uygulanır:

Konu	Yaklaşım	İlgili Bölüm
Loglama	log4j2 JSON layout → Kafka → Logstash → OpenSearch; yapılandırılmış, seviye/servis bazlı; gürültü bastırma (client-abort DEBUG)	§10
Hata yönetimi	Merkezi @RestControllerAdvice + tek tip ErrorResponse sözleşmesi	§4.6
Girdi doğrulama	DTO'larda @Valid/Bean Validation; ihlalde 400 + alan-mesaj haritası	§4.6, §8.3
Güvenlik	JWT doğrulama, filtre zinciri (sync/ban/oturum), rol bazlı yetki, IDOR koruması	§8
Önbellek & performans	Redis ile dış kaynak soyutlama; periyodik sync; düşük gecikme	§4.5, §4.7, §6.5
Eşzamanlılık & zamanlama	@Scheduled işler; asenkron Kafka üretimi; (dağıtık kilit yok — §9.4 notu)	§4.7, §9.4
Konfigürasyon	Profil tabanlı (5 profil) + ortam değişkeni; sır yönetimi env/K8s Secret	§4.8

Uluslararasılaştırma	Frontend i18next (TR/EN); backend haber çevirisi (Lingva); locale-duyarlı biçimlendirme	§5.6
Gözlemlenebilirlik	OTel 3-sütun; actuator sağlık/metrik uçları	§10
Eşleme (mapping)	MapStruct (entity↔DTO) + Lombok (boilerplate) — tüm modüllerde tutarlı	§4.1
API sürümleme	/api/v1 kök ön eki — tüm uçlarda	§7.1

12. Tasarım Kararları (ADR Kayıtları)

ISO/IEC/IEEE 42010 uyarınca, önemli mimari kararlar gerekçe ve ödünleşimleriyle kayıt altına alınır.

#	Karar	Gerekçe	Ödünleşim
ADR-01	Modüler monolit (tek Spring Boot, 19 alan modülü)	Düşük operasyon maliyeti, tek ekip, basit dağıtım; mikroservis karmaşıklığı bu ölçek için gereksiz	Tüm modüller birlikte dağıtılır/ölçeklenir; modül sınırları disiplinle korunmalı
ADR-02	Senkronize-et-ve-önbellekten-servis-et (circuit breaker yerine)	Kullanıcı isteği dış API beklemez; kırılğan/ücretsiz kaynaklar izole; düşük gecikme	Veri tazeliği TTL kadar gecikir; gerçek-zamanlı değildir
ADR-03	Kimliği Keycloak/OIDC'ye devretme	Standart, güvenli OIDC; 2FA/oturum/parola hazır; kendi kimlik altyapısı riskli	Ek altyapı bileşeni (Keycloak) işletme yükü
ADR-04	Çok katmanlı ban (SPI + filtre + oturum iptali + job)	Tek nokta atlatma riskini ortadan kaldırma; hem login hem API katmanı	Mantık birden çok yerde; SPI özel Keycloak imajı gerektirir
ADR-05	Symbol routing + Chart Strategy	Aynı varlık kaynaklarda farklı sembol/kaynak; tek tip akış için soyutlama	Çözücü/strateji bakımı; yeni kaynak = yeni strateji
ADR-06	Doğrudan repository injection (ayrı UoW/abstraction yok)	Sadelik; Spring/JPA zaten transaction yönetir; gereksiz soyutlamadan kaçınma	Servis-kalıcılık bağı sıkı (saf clean-arch yerine pragmatizm)
ADR-07	Kafka yalnızca log/olay akışı (→ Logstash → OpenSearch)	Asenkron, ölçeklenebilir, uygulamadan ayırık log toplama	Ek altyapı (Kafka/Zookeeper/Logstash)
ADR-08	/api/v1 sürümleme (context-path)	İleride v2'ye geçişte controller path'leri değişmeden kalır (32Bit isteği #18)	— (düşük maliyet)
ADR-09	git-SHA imaj etiketi + rolling update	İzlenebilir sürüm, kolay geri alma, sıfır-kesinti güncelleme	latest yerine SHA yönetimi
ADR-10	Sır yönetimi: env-var / K8s Secret	Güvenlik; kodda/git'te sır yok; ortam başına yapılandırma	Ortam kurulumunda env/secret yönetimi

ADR-11	LLM çoklu sağlayıcı failover (Gemini + Groq)	Ücretsiz katman kotaları ve erişilebilirlik; tek sağlayıcıya bağımlılık riski	İki sağlayıcı yapılandırması; davranış farkları
ADR-12	Lingva self-hosted çeviri	Ücretsiz, API key gerektirmez, hızlı (~200–500 ms); önceki LibreTranslate CPU'da 10–30 sn alıyordu	Self-host bileşen; Google'a dolaylı bağımlılık

13. Riskler

#	Kategori	Risk	Etki	Azaltım
R-01	Entegrasyon	Scraping kırılganlığı — İş Yatırım / Business Insider / halkarız / Truncgil sayfa yapısı değişirse ayrıştırma bozulur	Orta–Yüksek	Son-iyi snapshot + yedek hesap; izleme; client adaptörlerinde izolasyon (tek noktada düzeltme)
R-02	Entegrasyon	Ücretsiz API kota/limit (CoinGecko, Finnhub, FRED, Gemini/Groq) aşımı	Orta	Önbellek ile çağrı azaltma; LLM failover; günlük senkron sıklığı ayarı
R-03	Entegrasyon	Kaynak engelleme (Cloudflare/IP ban)	Orta	Matematiksel yedek (ör. Türk altını); alternatif kaynak
R-04	Veri kalitesi	Cache bayatlama — TTL içinde dış veri değişse de eski değer sunulur	Düşük–Orta	Uygun TTL; "bilgilendirme amaçlı" disclaimer; kritik veride kısa TTL
R-05	Ölçekleme	ShedLock yok — backend HPA ile >1 replikada @Scheduled işler her replikada çalışır (mükerrer çağrı / çift alarm bildirimi)	Orta	Mevcutta etkisi sınırlı (cache idempotent); kalıcı çok-replika için dağıtık kilit ekle (\$9.4)
R-06	Güvenlik	SPI fail-open — Keycloak DB okuyamazsa login'de ban kontrolü atlanır	Düşük	İkinci katman UserBanFilter API'de yakalar (çift katman tasarımı)
R-07	Bağımlılık	Keycloak / Lingva / LLM / SMTP erişilemezliği	Orta	Graceful degradation; LLM failover; çeviri TR fallback; alarm state korunur
R-08	Operasyon	Tek namespace / tek küme — altyapı tek nokta	Orta	Yönetilen GKE; health probe + otomatik yeniden başlatma; HPA

R-09	Yasal	Verilerin yatırım tavsiyesi gibi algılanması	Orta-Yüksek	"Yatırım tavsiyesi değildir" disclaimer (SPK); yalnızca bilgi sunma
------	-------	---	-------------	---

14. Tasarım Doğrulama (V&V)

Tasarım, çok katmanlı test ve statik analizle doğrulanır.

14.1 Test Stratejisi

Katman	Kapsam	Araç	Kanıt
Birim testi	Servis/iş kuralı mantığı (izole)	JUnit 5 + Mockito	~1000 @Test, 100 dosya
Entegrasyon testi	DB/uygulama bağlamı	@SpringBootTest + H2 (*IT.java, Failsafe)	Maven verify
Web/Controller testi	API uçları + güvenlik kuralları (ban/yetki)	MockMvc + spring-security-test	—
Frontend testi	Bileşen ve etkileşim	Vitest + Testing Library	91 test dosyası
Performans testi	Yük altında P95/hata oranı	k6 (scripts/perf-test.js)	P95 843 ms, %0 hata (SRS §7.2)
Statik analiz	Kod kalitesi + kapsam	SonarQube + JaCoCo	Kapsam raporu (CI artifact)

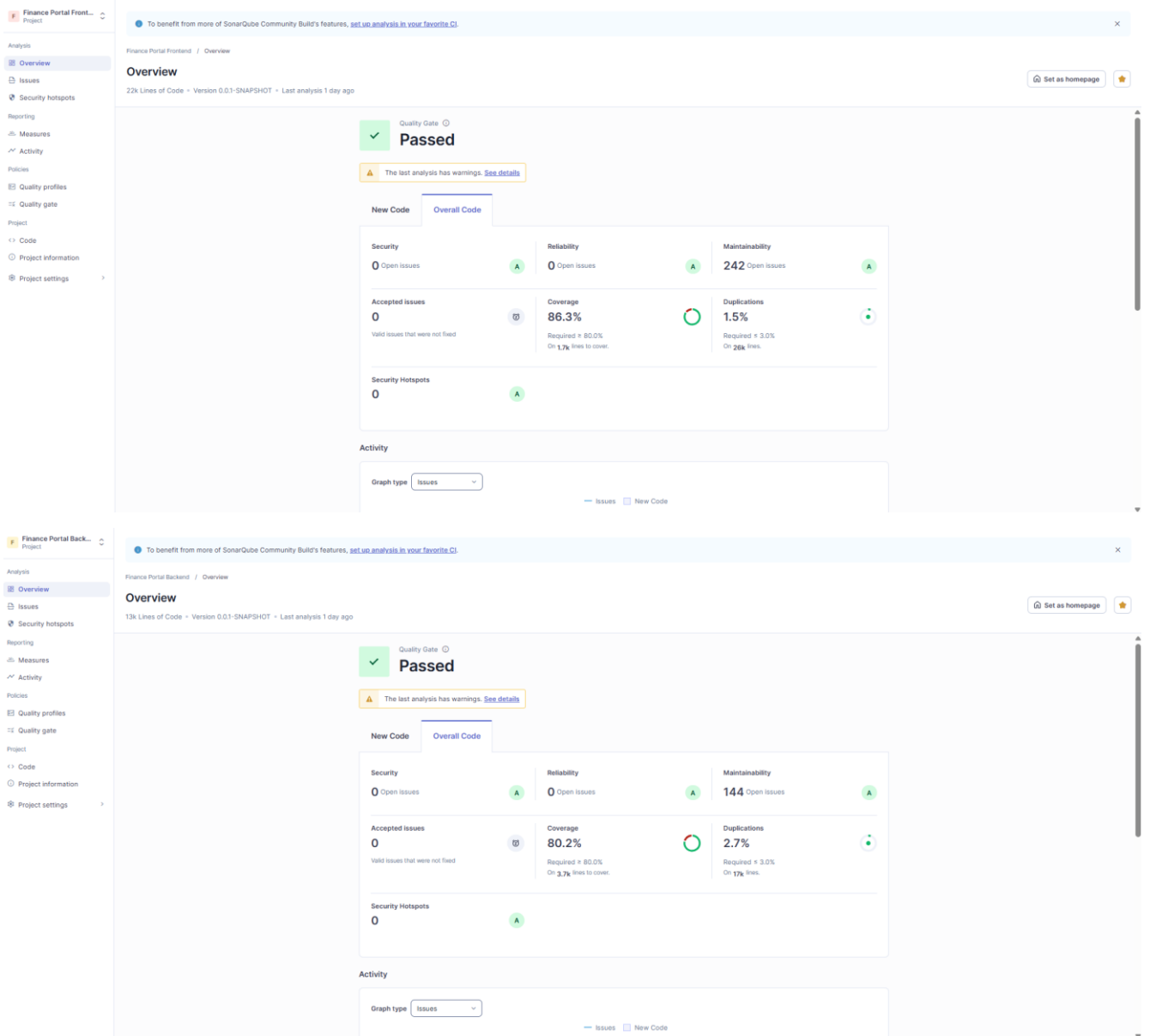
14.2 Doğrulama Yaklaşımı

Yön	Uygulama
Sürekli doğrulama	Her push/PR'de CI mvn verify (derleme + unit + IT + JaCoCo)
Kapsam dışı bırakma	DTO/entity/enum, config, client (scraping), MapStruct üretimi, SecurityConfig, Application — anlamlı test edilemeyen sınıflar kapsam metriğinden hariç (pom.xml)
Gereksinim izlenebilirliği	SRS §8.2'deki RTM ile fonksiyonel gereksinimler → testler eşlenir
Performans doğrulama	k6 + Grafana histogram_quantile (P95)
Manuel doğrulama	PR kod incelemesi + demo

Tasarım kararları da doğrulanır: ban (çok katman) güvenlik testleriyle, dayanıklılık (degrade) senaryo testleriyle, performans (cache deseni) k6 ile.

14.3 Kalite Kanıtı

Statik analiz ve kapsam, SonarQube + JaCoCo ile ölçülür ve raporlanır.



15. Ekler

Ek A — Teknoloji Versiyon Matrisi

Bileşen	Versiyon
Java	21
Spring Boot	3.3.9

springdoc-openapi	2.6.0
Keycloak Admin Client	24.0.1 (Keycloak ~24)
MapStruct	1.5.5.Final
OpenTelemetry Instrumentation	2.4.0 (+ Java Agent)
Flyway (plugin)	10.10.0
jsoup	1.17.2
YahooFinanceAPI	3.17.0
JaCoCo	0.8.11
JUnit / Mockito	5.9.3 / 5.x
React	19.2
Vite	8
react-router-dom	7.14
@tanstack/react-query	5.100
i18next / react-i18next	26 / 17
klinecharts / lightweight-charts / recharts	9.8.5 / 5.2 / 3.8
Tailwind CSS	4.2
Vitest	4.1
Altyapı	PostgreSQL · Redis · Kafka+Zookeeper · OpenSearch · Logstash · Prometheus · Tempo · Grafana · Lingva · cert-manager (k8s manifestlerinde sabitlenmiş)

Ek B — Paket / Klasör Haritası

Backend (com.otuzikibit.finance_portal): client/{binance,yahoo} · config · controller/{market,finance,user,alarm,chat} · domains/<19 alan>/{controller,service,client,dto,strategy} · exception · model/{entity,dto,enums} · repository · security · service/{auth,user,alarm,mail,messaging,market,chat/{llm,tools}} · util

Frontend (finance-frontend/src): pages/ (22) · components/{layout,charts,portfolio,news,chat,ui,...} · context/ · hooks/{,charts} · config/ · constants/ · i18n/locales/{tr,en} · utils/

Altyapı (k8s/): namespace · secrets · configs · data · messaging · auth · app · observability · scaling · network · keycloak-providers/ban-authenticator (SPI) · keycloak-image, keycloak-themes

Ek C — Açık Teknik Konular

#	Konu	Not
1	Dağıtık kilit (ShedLock)	Çok-replikalı @Scheduled için eklenmeli (§9.4, R-05)
2	WCAG 2.1 AA	Tam erişilebilirlik denetimi planlı (SRS §7.4)
3	Scraping izleme	Kaynak sayfa değişikliklerine karşı otomatik uyarı önerilir (R-01)
4	Tarihçe uçları	Bazı fetchHistory (fon/ViOP/eurobond) client'ta mevcut ancak ayrı uçla expose edilmemiş; gerektiğinde açılabilir

Ek D — 32Bit İster Uygunluk Matrisi

Bu tasarımın 32Bit teknik isterlerini nasıl karşıladığı:

32Bit İster	Tasarım Karşılığı	Bölüm	Durum
Container tabanlı (Docker imaj; Compose veya K8s)	Docker imajları + Docker Compose (baz) + Kubernetes/GKE (üretim)	§9.1, §3.2	✓(aşıyor)
Backend Java/Spring Boot, REST, katmanlı, config, exception + standart yanıt	Spring Boot katmanlı; /api/v1; GlobalExceptionHandler+ErrorResponse; profil/env config	§4, §7.1	✓
Frontend React: component, routing, state, REST	React 19; react-router; react-query+Context; axios	§5	✓
PostgreSQL, normalize, JPA/Hibernate, migration	PostgreSQL + Spring Data JPA + Flyway (V1–V21)	§6	✓
Loglama log4j2; OpenSearch'e (Kafka→tüketici); JSON alanları	log4j2 JSON → Kafka → Logstash → OpenSearch	§10, §11	✓
OpenTelemetry: metrik + trace/span + dashboard	OTel Agent + Micrometer/Prometheus + Tempo + Grafana	§10	✓
Keycloak + LDAP, RBAC, 2FA/OTP, remember-me, SSO/JWT	Keycloak/OIDC + OpenLDAP; USER/ADMIN; TOTP; rememberMe; JWT	§8	✓
Performans < 2 sn	P95 843 ms (ölçülen)	§14, SRS §7.2	✓
Dokümantasyon (mimari + API listesi)	Bu SDD + Swagger/OpenAPI + SRS §4	§7.1	✓
Test (unit/integration)	~1000 @Test + IT + 91 FE testi	§14	✓
Basit CI/CD pipeline	GitHub Actions CI + CD (GKE)	§9.5	✓(aşıyor)
Config'den API URL; scheduler+cache; Redis	external-api.* config; @Scheduled; Redis	§4.7, §4.8, §6.5	✓

TEKNİK TASARIM DÖKÜMANI SONU. Bu doküman IEEE 1016-2009, ISO/IEC/IEEE 42010, C4 ve OWASP ASVS referans alınarak hazırlanmış; tüm tasarım kararları mevcut kod tabanından doğrulanmıştır. Gereksinim tanımları için **Analiz Dökümanı (SRS)**'ne bakınız.